Candidate Name: Sam Thomas Candidate Number:

# OCR GCE A COMPUTER SCIENCE PROJECT H446-03

Name: Sam Thomas

Candidate Number:

Archway School: Callywith College

Title of Project: The Final Bonfire

# Table of Contents


Analysis ..... 6
Project Definition ..... 6
Setting the Scene ..... 6
Stakeholders ..... 7
Identifying the target market and suitable representative ..... 7
Using computational methods ..... 7
Thinking Abstractly ..... 7
Thinking Ahead ..... 7
Thinking Procedurally ..... 8
Thinking Logically ..... 8
Thinking Concurrently ..... 8
Research ..... 9
Interview with Ed White ..... 9
Interview Script ..... 10
Review of Interview ..... 11
Research into FromSoftware's Dark Souls and similar games ..... 12
Features of the proposed solution ..... 18
Identifying the features going to be used ..... 18
Limitations of the features ..... 20
Realistic overview ..... 20
Hardware and Software Requirements ..... 20
Hardware requirements for intended platform ..... 20
Software requirements for The Final Bonfire ..... 22
Extra Utilities ..... 23
Success Criteria ..... 23
Design ..... 26
Structure of the game ..... 26
Systems diagram ..... 26
Breaking down the problem systematically ..... 27
Decomposition of each component ..... 27

Interface Designs ..... 29
Input Validation ..... 33
Justification for approaches ..... 33
Algorithms ..... 33
Flowcharts for the broken-down problems ..... 33
Data Structures ..... 44
Class diagrams ..... 44
Justification of the class diagrams ..... 47
Formatting ..... 50
Testing ..... 50
Testing Table for Success Criteria ..... 50
During/ Post development tests ..... 52
Development ..... 54
Assets ..... 54
Navigational Home Screen ..... 55
Start and Exit game ..... 56
Sound ..... 57
Controls Screen ..... 58
Loading Screen ..... 60
Review of the home screen ..... 60
Levels ..... 62
Review of the levels ..... 64
Character Movement ..... 65
Idle, Walking, Running ..... 65
Jumping ..... 68
Roll ..... 70
Review of the character movement ..... 72
Health and Stamina Bars ..... 72
Consultation about health script ..... 72
Health ..... **Error! Bookmark not defined.**
Stamina bar ..... **Error! Bookmark not defined.**
Review of the health and stamina bars ..... **Error! Bookmark not defined.**

Interactions ..... **Error! Bookmark not defined.**
NPC ..... **Error! Bookmark not defined.**
Checkpoint(s) ..... **Error! Bookmark not defined.**
Health Increase ..... **Error! Bookmark not defined.**
Stamina Increase ..... **Error! Bookmark not defined.**
Multiple checkpoints ..... **Error! Bookmark not defined.**
Review of the interactions ..... **Error! Bookmark not defined.**
Enemies ..... **Error! Bookmark not defined.**
Bat ..... **Error! Bookmark not defined.**
Review of the enemies ..... **Error! Bookmark not defined.**
Reflection ..... **Error! Bookmark not defined.**
Evaluation ..... **Error! Bookmark not defined.**
Testing ..... **Error! Bookmark not defined.**
Testing of entire game/ success criteria ..... **Error! Bookmark not defined.**
Evidence of testing ..... **Error! Bookmark not defined.**
Usability features ..... **Error! Bookmark not defined.**
Breaking down features ..... **Error! Bookmark not defined.**
Main Menu ..... **Error! Bookmark not defined.**
Controls screen ..... **Error! Bookmark not defined.**
Loading Screen ..... **Error! Bookmark not defined.**
Upgrade Screen ..... **Error! Bookmark not defined.**
Stakeholder testing/ review ..... **Error! Bookmark not defined.**
Stakeholder feedback analysis ..... **Error! Bookmark not defined.**
Remaining Success Criteria ..... **Error! Bookmark not defined.**
Enemies and Bosses SC 9,15,23,24,26 ..... **Error! Bookmark not defined.**
Damage upgrade SC 12 ..... **Error! Bookmark not defined.**
Item bar SC 13,27 ..... **Error! Bookmark not defined.**
Flames SC 14 ..... **Error! Bookmark not defined.**
Rolling invincibility SC 16 ..... **Error! Bookmark not defined.**
Health flasks SC 17,18,19,20 ..... **Error! Bookmark not defined.**
Death process SC 21,22 ..... **Error! Bookmark not defined.**
Cutscenes SC 29 ..... **Error! Bookmark not defined.**

Overall review of the project ..... **Error! Bookmark not defined.**
Code Overview ..... **Error! Bookmark not defined.**
Character ..... **Error! Bookmark not defined.**
Movement ..... **Error! Bookmark not defined.**
Health ..... **Error! Bookmark not defined.**
Stamina ..... **Error! Bookmark not defined.**
General ..... **Error! Bookmark not defined.**
Side Scrolling Camera ..... **Error! Bookmark not defined.**
Interaction ..... **Error! Bookmark not defined.**
Character interactions ..... **Error! Bookmark not defined.**
NPC ..... **Error! Bookmark not defined.**
Checkpoint ..... **Error! Bookmark not defined.**
Upgrades ..... **Error! Bookmark not defined.**
Tests ..... **Error! Bookmark not defined.**
TextMeshPro calling test ..... **Error! Bookmark not defined.**
UI ..... **Error! Bookmark not defined.**
Loading Screen ..... **Error! Bookmark not defined.**
Main Menu ..... **Error! Bookmark not defined.**

# Analysis

For references I have coloured the words **Orange**

## Project Definition

### Setting the Scene

In my project I will be creating a Pixel 2D game inspired by the popular game collection '**Dark Souls**' by the game developers **FromSoftware**. This is a difficult 3rd Person RPG where you fight bosses and follow a detailed story line to 'Relink the fire'. When the character kills bosses or enemies they pick up items called 'souls' these can be used to upgrade the characters stats, I will implement this into my game. Throughout the game you stop at checkpoints known as bonfires, I am going to target my game around the idea that there is only one fire left in a deep cave guarded by the final boss known as the Protector of the Flame and the aim is to harness the final flame and wreak chaos on the world and start a new chapter of the games story. The Main Character will go through 3 levels, each with increasing difficulty until the final boss. Similar games also in the 'souls-like' genre could be **Hollow Knight** created by **Team Cherry** or **Zelda II: The Adventure of Link** by **Nintendo**.

In order for me to do this the game will require: Home screen with UI; Item wheel for healable flask, items for removing debuffs, item for giving buffs; Main character; Enemy characters; NPCs; Bosses; Interactive checkpoints; interconnected scenes/ maps; Death Screen; Reward items for killing enemies; 3 identifiable maps; Health Bar; Stamina bar; Upgrade points for character health and damage to increase. This will all be researched deeply in order for me to find the best way to do each part.

# Stakeholders

## Identifying the target market and suitable representative

The game will be based around the pixel design so cannot be associated directly with real life, no gore so cannot be limited to age 15 and above, includes fantasy characters that may be deemed as 'scary' so typically age 13 and up. The game will also include a high difficulty game aspect (as souls-likes are known for their extreme difficulty) the characters also use weapons such as swords or sickles which could be deemed inappropriate for smaller children. With all this in mind I think the game will be targeted at age 13 and above, or in terms of the **PEGI** system it would be PEGI 12 (**| Pegi Public Site**) . I have chosen a friend to become part of my target audience – Ed White, he is a 16-year-old student at Callywith that greatly enjoys the kind of game I am creating having played many soulslikes himself. I have many chances to talk to him as I spend most of my free time with him. He will be working closely with me and I will get him to help with his opinions wherever possible, as I will be following a RAD approach to my development of the game and will get possible ways to make my game better from him after each cycle of my development.

# Using computational methods

My game is suited to being created on a computer as I can use computational methods to solve the various problems surrounding the game. This is simple to do as a computer can easily remove the concept of real life even using simple ideas and algorithms that can be designed and created for my game. Below I have specified how this is true using computational methods.

## Thinking Abstractly

Abstraction is a process where you omit superfluous details and focus only on relevant ones, I can do this by isolating basic game mechanics, UI and other game components inside of my game. For example, I could use minimalistic ideas for my game UI and only include the base of what is needed such as Play and Exit. This helps game development by saving time and getting the base of the game completed before extras are added.

## Thinking Ahead

Thinking ahead is the ability to plan and foresee future implications, requirements and obstacles. My game has various possible implications and I have extra ideas of how I could improve my game given extra time to complete this. For example, I know that I will have to learn how to employ certain game mechanics such as a side scrolling screen, which I can plan to leave extra time for me to do, or learn it along side creating the base game. This helps the development of the game by saving massive amounts of time

redoing and editing items which could already be foreseen as a problem. However, certain factors may be unforeseen and will have to be changed and edited, costing time.

## Thinking Procedurally

Procedural thinking is to understand and implement the games development as a series of steps or procedures. I can do this by planning out my development of the game's algorithms and functions into various steps which make sense to the way I will develop my game. For example, I will always declare all global variables at the top of the code which I can refer to, and all the character and enemies' movement will all be coded exactly the same so there will be no confusion for me later on. This is useful for development as it opens the possibility of reusable code and employs organized and efficient code. However, this could create the possibility of reusing ineffective code so I will have to make sure that my code is perfect for one item for it to be reused.

## Thinking Logically

Logical thinking is consistently using reason to solve problems and arrive at a valid, effective conclusion. This can be applied to my game through solving errors within my code, or problems with my assets. For example, I could create conditional statements within my code for different states or elements inside the game. Logical thinking helps game development by aiding the creation of viable, error-free, efficient code, but this comes at the cost of difficulty in maintaining this logical consistency, potentially causing other problems.

## Thinking Concurrently

Thinking concurrently is the ability to multitask (managing various tasks, processes or threads all at the same time). This can be implemented into my game as it will have to handle multiple inputs entered at the same time, while loading various aspects of the game and handle the enemy's movement as well. For example, I could manage creating the game as well as learning new code like I specified before, or the computer could manage multiple user inputs at the same time. This is very helpful for the game as it allows the game will run smoother and faster meaning there will be less delay between inputs. Challenges are also present such as conflict between concurrent tasks attempting to be executed at the same time.

# Research

## Interview with Ed White

For the interview I created a questionnaire (With help of **AI: ChatGPT** for more ideas) to have fast and important questions which I think is vital for the main aspects of the game.

**Gameplay Experience:**

- What kind of combat mechanics do you enjoy in Soulslike games, and why?
- What kind of game aspects do you like such as checkpoints and items?

**World and Level Design:**

- What elements do you find most engaging in the world design of Soulslike games?
- Are there any specific environments or settings you would love to explore in a game like this?

**Enemy Design and AI:**

- What do you think makes for a compelling and challenging enemy in a Soulslike game?
- How important is enemy variety to you?

**Additional Features:**

- Are there any unique features or mechanics you think would enhance the Soulslike experience?

**Overall Impressions:**

- What do you think are the most essential elements for a successful Soulslike game?
- How do you feel about the pacing and flow of these games?

# Interview Script

(A = Answer)

**Gameplay Experience:**

- What kind of combat mechanics do you enjoy in Soulslike games, and why?
    A. I enjoy the avoiding mechanics (rolling and jumping), because it adds an energetic aspect to combat which keeps you on your toes
- What kind of game aspects do you like such as checkpoints and items?
    A. I like having regular checkpoints (such as bonfires and alters) in these types of games so that the gameplay is smooth and isn't too challenging to get all the way through. Healing items (like flasks) is also helpful for the same reason.

**World and Level Design:**

- What elements do you find most engaging in the world design of Soulslike games?
    A. I find boss fights most engaging; it provides a singular large challenge which gives a game a motivating aspect to keep players persistent
- Are there any specific environments or settings you would love to explore in a game like this?
    A. I think that cave settings provide a close quarters environment for the player, this entails fast paced gameplay and combat which I find very engaging.

**Enemy Design:**

- What do you think makes for a compelling and challenging enemy in a Soulslike game?
    A. I think for this type of game, enemies should be quite challenging to enforce a strong mindset for the player and keep them engaged
- How important is enemy variety to you?
    A. It's important to have different enemies in different areas of the game so that the game doesn't become boring or levels become undefinable

**Additional Features:**

- Are there any unique features or mechanics you think would enhance the Soulslike experience?

    A. I'd like to see status effects such as poison or bleed, and a way to nullify these as it adds more variety to the game and increases the difficulty factor which I believe is what soulslike games are all about.

**Overall Impressions:**

- What do you think are the most essential elements for a successful Soulslike game?

    A. I think that a soulslike game should be difficult overall and require hard work so that it does not become boring, I also think that these games should include character progression such as levelling up and upgrades to make the player feel as if they are making progress through the game.

- How do you feel about the pacing and flow of these games?

    A. These games should be moderately fast, and levels should flow cleanly into one another to provide stable and enjoyable gameplay.

## Review of Interview

I would say this interview gave me a greater understanding of what people and my stakeholder will want to see in my game.

I agree with all that was mentioned in the answers provided apart from the second bullet point about regular checkpoints, which I believe may make the game too easy for what I'm trying to create, however, I will attempt to take this opinion as a way to make the game better so I will create slightly more than I was anticipating but not too many that it makes the game too easy and boring to play.

I like the idea of including the status effects and completely agree that it will add more variety to the game as I do not want it to be too simplistic or boring to play due to similar gameplay throughout.

I agree with the idea that enemies should be challenging and I will have to think hard about how I can make the enemies difficult and different from each other, all the while keeping the game challenging but not similar in fights as it should be different for each level to enforce the idea of moving through separate levels up to the final boss (which was also mentioned by the stakeholder under the same -Enemy Design- heading)

## Research into FromSoftware's Dark Souls and similar games

**FromSoftware's Dark Souls** is a 3rd person RPG where the player goes through the games various 'levels' (Different areas of the game) and fights many incredible bosses to complete the end goal of dethroning the Lord of Cinder and linking the first flame. Throughout the game you upgrade your character using bonfires as well as using them as checkpoints to respawn at. You use items such as healing flasks to heal your character in battle or remove long lasting effects like poison. There are many basic enemies in the game as well as various 'mini bosses' (High health and damaging enemies that are not classed as bosses) which make your playthrough just that bit more difficult.

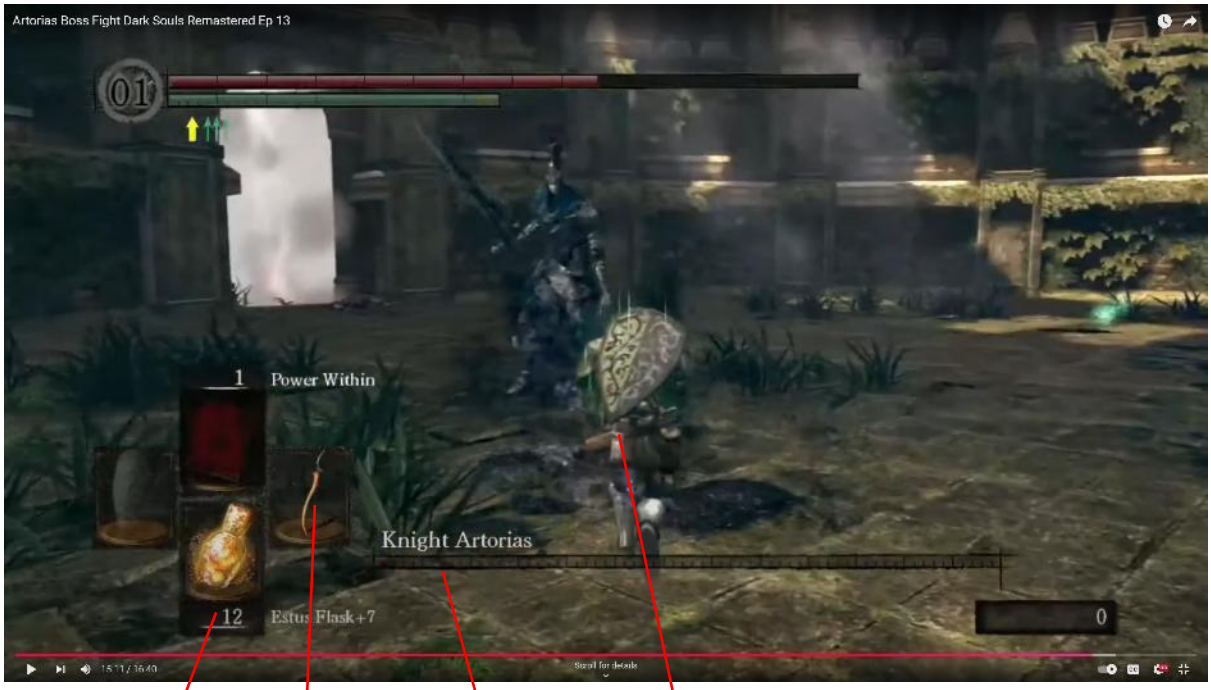
### Fighting Bosses

Character's healthbar and staminabar

Boss

**Credit: Sway Realm Games (YouTube)**

**Artorias Boss Fight Dark Souls**




Healable flask item

Weapons

Boss name and healthbar

Character

Boss arena which character has to enter to fight the boss, cut off by 'fog gates' (The white fog in the doorway) which the character cannot leave once entered until you die or the boss is killed

**Credit: wallpaper safari**

**Free Download Dark Souls Bonfire Wallpaper by @joshuaf64 | WallpaperSafari**

## Using Bonfires

When you rest at bonfire your health is replenished to the maximum and all enemies reset back to where they spawn in




Various options for what the character can do – most notably being the level up option which I will add to my game

Bonfire that you interact with to rest at

Another game that has the exact art style I'm attempting is **Blasphemous** created by **The Game Kitchen**. It is a turn-based pixel 2D platform game following the level design of dark souls games. It follows with the idea of using flasks and using spells and weapons to fight enemies, there are also various ways to upgrade your character and build resistance to effects. Also 'bonfires' are used but in the form of altars located around the map.

## General Gameplay Image

Credit: Alessandro Fillari (Game Spot) (Image only)

Blasphemous Review - Darker Souls - GameSpot

Health bar and Stamina Bar




Flasks and how many uses

Character

Dialogue

Enemy

Background

**Credit: Ykrad (YouTube)**

**Blasphemous 100% Completion Walkthrough - Part 01 [PC/Steam] (Read the description for Timestamp)**

## Altar Description




Information on how the checkpoints work

Images depicting how you know you have saved your checkpoint

After researching deeply into these games, I am going to take aspects from both and use in my opinion the best ideas both games involve all the while keeping it within the 2D gameplay plan. For example, I have decided the best 'checkpoint' aspect is from **Dark Souls** as it promotes a more in-depth interface and uses good ideas for upgrades which is important for my game as it shows progression throughout and does not feel like you haven't improved your character moving through the game.

Both these games also use short but captivating and emotional cutscenes which I will include in my game to create a greater feeling of immersion and potentially deepen the short storyline I will add. I feel these cutscenes really make a lot of the game for me so I will be sure to implement it best as possible.

**Dark Souls Intro cutscene**

**Credit: Zozo Bozo**

**DARK SOULS REMASTERED Intro Cutscene FULL HD 60 FPS**




**Blasphemous Cutscene after boss fight**

**Credit: AllGamesWorldHd**

**Blasphemous Digital Deluxe Edition Gameplay (PC Game)**



The games also include death screens/ titles that indicate when you die, this adds a more soulslike feel as it is common for games to recreate this idea with the most iconic

being from **Dark Souls 1**. Many ‘memes’ and separate videos and images outside of the game have used this death screen, so I think it is important to implement a different version of this into my game and if my game was to be released I think it would help make the game more relevant.

**Credit: Gamesradar**

**How FromSoft took the action-RPG to a new level by putting faith in Dark Souls' players | GamesRadar+**




# Features of the proposed solution

## Identifying the features going to be used

I have discussed many ideas and features from these soulslike games, so these are the ones I will use including extras I have thought of.

| | |
|---|---|
| Checkpoints | Allow the player to rest/ replenish flasks, upgrade character. Makes the game easier and more forgiving. |
| Upgrade menu | Allow the player to upgrade the character with flames to create a feel of progression and makes the game more interesting. |
| Healing flask count | Indicates how many flask uses the player has left so the player can work out how to use them most effectively. Makes the game more difficult but more engaging to play |

| | |
|---|---|
| Status effects | Status effects of poison and bleed, poison will be curable bleed will not. Makes the game more strategic and interesting as there is more to look out for. |
| Items | Items such as a cure to the poison status effect which the player can use to cure themselves. Creates more strategic approach to the game allowing for more intense gameplay. |
| Multiple enemies and bosses | The game will feature different enemies located in different locations and levels, as well as bosses at the end of each level. This makes the game more complex and engaging. |
| Start menu | Start menu with Play, Exit and mute music. Giving the player more options on the menu screen to create better ease of play. |
| In game UI | Will include the items you can use, your health and stamina, amount of flames held. |

| | |
|---|---|
| Flames | Similar to 'souls' from **Dark Souls**, it is an item picked up from enemies after you kill them, can be used to upgrade your character. Makes the game feel more complex and engaging for the player to keep these flames as they will disappear upon death. |
| Updated boss UI | When player enters a boss room the bosses health and title appear at the bottom of the screen. This indicates you are fighting a boss and makes the game more simplistic as you can see the bosses health bar. |
| Level transitions | Entering into the next level without any loading screens. Makes the game feel smoother. |
| Separate levels | Creates a feeling of progression through out the game. |
| Sound/ music | Sounds and music will be included to make a more pixel-like and a greater feeling immersion into the game. |
| Keyboard and mouse control | This will be easy to play as not many keys are included in the game but each have important functions. |
| Cutscenes | After bosses have been killed, cutscenes will play to create an idea of a story and increase the complexity of the game. |
| Death screen | Death screen to indicate that the character has died. |

## Limitations of the features

My ideas and features have various time, hardware and software limitations.

All the features hold the same limitation of time, as I take 2 other A-Levels that also have a lot of work required in general. Other limitations are the entire asset and visual aspect of the game, due to the fact I have had many new ideas since starting this project I will be required to find new icons and assets for features like 'Flames' and 'Status effects', and I potentially may not actually find any viable assets so I will have to create them myself. Another is I am required to find someone to create and animate a cutscene for me as I am not good at animating, and it will be extra work to attempt to find similar assets for the animation in a 3D version as I think it would be good to create the idea of it being a 3D world while having the actual gameplay in 2D. A final limitation would be generally learning how to create all the new things inside my game, for example the level transition and moving background part as I cannot currently code that myself, but I will learn how to within the timeframe.

## Realistic overview

Realistically I believe all of this can be done within the timeframe, or at least the very large majority as there are not too many new things for me to learn. Due to the fact I think this can be done with time left there is a potential opportunity for me to make the game even more complex such as with new status effects, but being realistic this is only a possibility as I am aiming to get everything I have mentioned fully done without adding extra work for myself until it's possible for extra work to be done.

# Hardware and Software Requirements

## Hardware requirements for intended platform

This game is intended to be for PC (**Windows**) only as I do not have the hardware possibility of using a controller to add in PlayStation and Xbox.

**Hardware requirements for Windows 11** (Windows 11 Specs and System Requirements | Microsoft)

| | |
|---|---|
| **Processor** | 1 Gigahertz (GHz) or faster |
| **RAM** | 4 Gigabytes |
| **Hard Drive size** | 64 or larger |
| **Graphics card** | **DirectX 12** or later with WDDM 2.0 driver |
| **Display** | High definition (720p) display that is greater than 9" diagonally |

| | |
|---|---|
| **Internet connection** | Stable internet for consistent updates and new features |

**Estimated Hardware requirements for The Final Bonfire** (Blasphemous system requirements | Can I Run Blasphemous)

| | |
|---|---|
| **CPU** | **Intel Core2** Duo E8400 or AMD Phenom II x2 550 |
| **RAM** | 2 GB |
| **Graphics card** | **GeForce GTX 260** or **Radeon HD 4850** |
| **Dedicated video RAM** | 512 MB |
| **Keyboard** | Control character inside the game |
| **Mouse** | Navigate game UI |
| **Monitor** | Relay what is happening onto the screen so the user can see it |
| **Speakers** | Play sound back to create immersion |

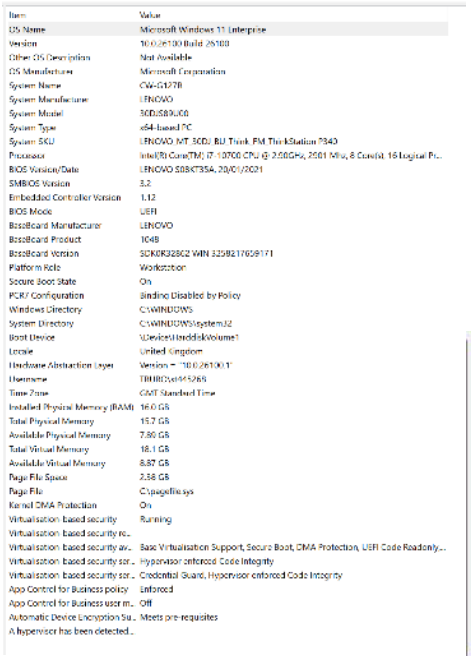
**Unity Player Requirements (For Windows 10 and higher)**

| | |
|---|---|
| **Operating system ver.** | **Windows 10** version 21H1 (build 19043) or newer |
| **CPU** | x86, x64 architecture with SSE2 instruction set support, ARM64 |
| **Graphics API** | **DX10, DX11, DX12** or **Vulkan** capable GPUs |
| **Devices** | Workstation or Laptop form factors |
| **Keyboard** | Putting inputs into the game |
| **Mouse** | Navigating UI parts of the game |
| **Monitor** | Relay what is happening onto the screen so the user can see it |
| **Speakers** | Play any sound aspects in the game |

**Unity Editor Requirements (For windows 10 and higher)**

| | |
|---|---|
| **Operating system ver.** | **Windows 10** version 21HI (build 19043) or newer |
| **CPU** | X64 architecture with SSE2 instruction set support, ARM64 |
| **Graphics API** | **DX10, DX11, DX12** or **Vulkan** capable GPUs |
| **Devices** | Workstation or Laptop form factors |
| **Keyboard** | Writing out code, changing names of files etc. |
| **Mouse** | Navigating **Unity** Editor |
| **Monitor** | Relay what is happening onto the screen so the user can see it |
| **Speakers** | Any sound aspects |

The computers at my school succeed these requirements as seen here:






## Software requirements for The Final Bonfire (Editing and playing)

Only extra requirement for **Unity** Editor is hardware vendor officially supported drivers

**Software requirements for Unity Player (Windows)**

| | |
|---|---|
| **Drivers** | Hardware vendor officially supported drivers |
| **Development** | IL2CPP scripting back-end requires Visual Studio 2015 with C# Tools component or later and **Windows 10** SDK |

C# is the coding language that has been chosen as it is simple to learn, high in demand so will be useful for future projects, provides libraries to help with writing code and making it significantly faster, easily readable and understandable for looking back and re-writing code and is a flexible language to use and can be applied to other consoles and devices in case I want to add an extra platform later in development. C# is

extremely useful and resonates well with me as **Unity** uses C# and I have prior experience using both **Unity** and C# together and separately.

### Extra Utilities

I will need to have access to the internet, for finding information and getting assets. There will need to be access to files for me to navigate, edit file location and open applications. I will need to have visual studio installed to allow me to input code and access the library within it.

## Success Criteria

| No. | Requirements for game | Justify | Reference |
|---|---|---|---|
| 1. | Start up home screen | Allows user to choose whether to start or exit the game | Feature from **Dark Souls** |
| 2. | Create sound setting | Turn off/on sounds and music | Feature from **Dark Souls** |
| 3. | Create loading screen | Loading screen to indicate the game loading | Feature from **Dark Souls** |
| 4. | Create level design | Gives the user levels to progress through | Interview with Ed White |
| 5. | Add character with run, jump, attack, roll and interact aspects binded to keys on keyboard | Gives the user a character to use throughout the game | Feature from **Dark Souls**, Interview with Ed White |
| 6. | Create a NPC to tell your fate | Indicates what the player must do to complete the game | Feature from **Dark Souls** |
| 7. | Add working checkpoints | Gives the user a place to rest, will set spawn point interacted with, upgrade character | Interview with Ed White |
| 8. | Create checkpoint UI screen | Allows the user to navigate through the checkpoint to upgrade character and rest | Interview with Ed White |

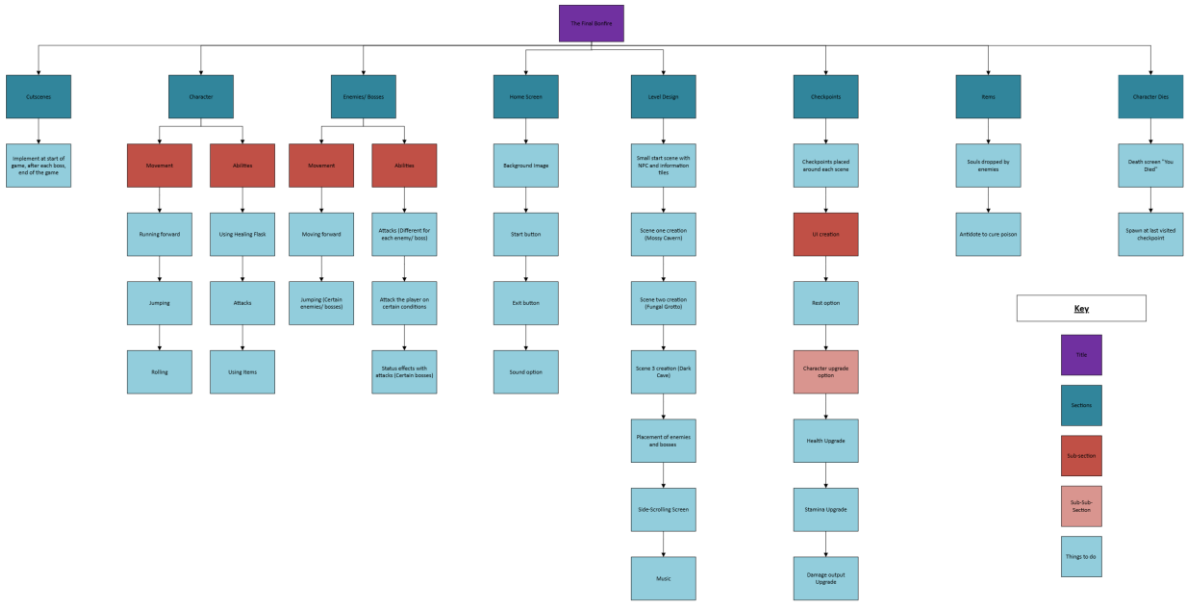
| | | | |
|---|---|---|---|
| 9. | Add enemies and boss to each level | Gives the user a challenge, makes the game more engaging | Feature from **Dark Souls**, Interview with Ed White |
| 10. | Add working health bar | Indicates the health the player is on | Feature from **Dark Souls** |
| 11. | Add working stamina bar | Indicates how much stamina user has, will regenerate over time, decreases with rolling and attacking | Feature from **Dark Souls** |
| 12. | Add working upgrade screen for health, stamina and attack | Creates feeling of progression and increases the complexity and variety of game mechanics | Interview with Ed White |
| 13. | Add item bar | Shows the user what items they have e.g. item to stop poison | Interview with Ed White |
| 14. | Add 'flames' number on the screen | Indicates how many flames are held by the user which they can use to level up the character | Interview with Ed White |
| 15. | Create enemies that will attack the user using random attacks | Makes the gameplay more challenging and increased variety so the user wont be able to predict attacks as they will be random | Interview with Ed White |
| 16. | Make it so when the character rolls, they are invincible | Gameplay becomes more engaging and strategic | Interview with Ed White |
| 17. | Create flasks for the character to regain health with | Gameplay becomes more engaging and strategic | Interview with Ed White |
| 19. | Create flask count on the screen | Indicates how many flasks are held by the user | Feature from **Dark Souls** |
| 20. | Make it so the flasks get reset when checkpoint is rested at, or the character respawns | Makes the gameplay more forgiving | Feature from **Dark Souls** |

| | | | |
|---|---|---|---|
| 21. | Create an item that allows the user to upgrade flask count | Increases gameplay variety | Feature from **Dark Souls** |
| 22. | Create death screen for when the user dies | Indicates to the user that they have died | Feature from **Dark Souls** |
| 23. | Make the user respawn at the last accessed checkpoint | Makes the game less tedious and creates idea of progression | Interview with Ed White |
| 24. | Scale bosses to be larger | Creates a more commanding and deadly presence for the boss | Feature from **Dark Souls** |
| 25. | Add boss health bar and name when entering boss area | Indicates how much health the boss has the bosses name, makes gameplay more engaging | Feature from **Dark Souls** |
| 26. | Make a side scrolling screen | Makes the gameplay smoother flowing and better to play as the character stays in the middle of the screen mostly | Interview with Ed White |
| 27. | Implement status effects and apply to different bosses | Makes the game more challenging and increases variety so the game does not get boring | Interview with Ed White |
| 28. | Create items to nullify status effects, but in limited supply and hard to find | Makes the gameplay more strategic and high risk – high reward as there will only be a limited supply of the items | Interview with Ed White |
| 29. | Add different music for each level | Makes the gameplay more immersive and creates a better atmosphere surrounding the game | Feature from **Dark Souls** |
| 30. | Implement cutscenes after bosses and for the end of the game | Makes the game more immersive and enticing | Feature from **Dark Souls** |

# Design

## Structure of the game

### Systems diagram




Why use a system diagram?

I used a system diagram as it allows me to develop a mental structure of how I am going to create my game. It allows me to use the computational thinking skill “abstraction” meaning I can create and visualize the base game (the higher up light blue boxes) and then add to it later on (lower down light blue boxes). It also allows me to use the computational thinking skill “thinking ahead” to determine where potential errors could occur and which sections to put more time into, for example I can tell from looking at the diagram I will need to put considerably more amounts of time into the Level Design than items. Finally, I can also utilize the computational thinking skill “thinking logically”

which helps me understand where decisions will be necessary and what problems it could cause when developing the game. All of this will give me the best chance of completing the development of the game using good structure and maximum efficiency.

## Breaking down the problem systematically

### Decomposition of each component

**Cutscenes**

Implementing the cutscenes into various areas of the game such as after each boss.

**Character**

**Movement**

Running forward: Allows the player to move the character forward.

Jumping: Allows the player to make the character jump.

Rolling: Allows the player to make the character roll to avoid getting hit by enemies.

**Abilities**

Using healing flasks: Allows the player to heal themselves with a healing flask.

Attacks: Allows the player to attack the enemies with various attacks.

Using items: Allows the player to use items such as an antidote to cure themselves.

**Enemies/ Bosses**

**Movement**

Moving forward: Different from character movement as some enemies float, makes the enemies move.

Jumping: Makes certain enemies and bosses be able to jump.

Death: Play death animation and remove when character kills an enemy. (Addition)

**Abilities**

Attacks: Makes the enemies be able to attack the player.

Conditions: Give the enemies conditions on when they can attack the player such as when they are in proximity to them.

Status effects: Make certain bosses and enemies have certain attacks that apply the player with status effects.

**Home Screen**

Background Image: Create a background image for the home screen, making it look better.

Start button: Allows the player to enter the game by pressing start.

Exit button: Allows the player to exit the game.

Sound option: Gives the player the option to turn off the sound/ music.

**Level Design**

Start scene with NPC: Gives the player the start scene which will explain what they must do.

Scene one: Creation of the “Mossy cavern” Level.

Scene two: Creation of the “Fungal grotto” Level.

Scene three: Creation of the “Dark cave” Level.

Placement enemies/ bosses: Adding enemies and bosses to the levels.

Side scrolling screen: Makes the screen scroll with the player as they progress through the level.

Music: Add different music for each level to create greater immersion.

**Checkpoints**

Checkpoint placement: Adding checkpoints to each level.

**UI creation**

Rest option: Allows the character to “rest” and regain all health and healing flasks.

**Character upgrades option**

Health: Upgrade/ increase the amount of health the character has.

Stamina: Upgrade/ increase the amount of stamina the character has.

Damage: Upgrade/ increase the amount of damage the character outputs.

**Items**

Souls/ **Flames**: Pick up souls after killing an enemy – used for upgrades.

Antidote: Used to cure the poison status effect.

**Character dies**

Death screen: Upon death, the “You died” death screen will appear.

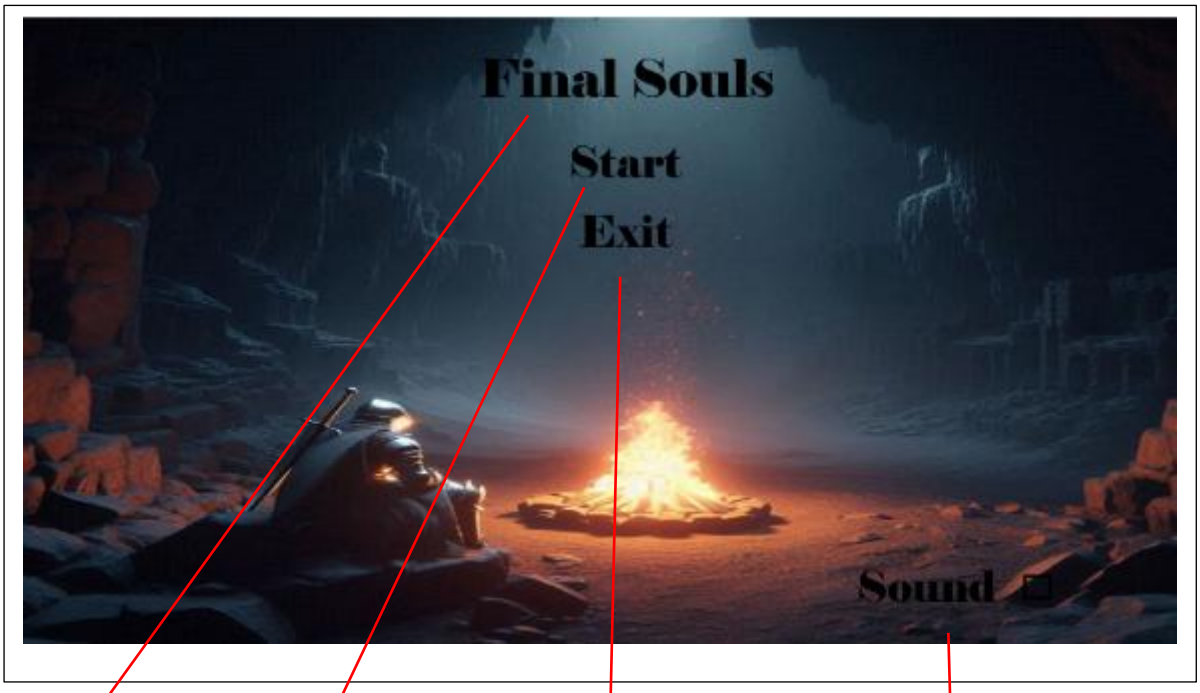
Respawn: Makes the character respawn at the last visited checkpoint.

## Interface Designs

Below I will provide potential ideas for what my user interfaces could look like on the base level.

All images are created using **AI: Copilot, Adobe Photoshop and Microsoft Paint**

**Home Screen**




Title, indicates the game they are playing

Start button to enter the game

Exit button to exit the game

Sound tick box to turn sound on/ off

I have used large fonts and included the text in the brightest part of the image so the user can clearly see the words and understand what they can do by using simplistic language.

**Text/ Information Box**




Where text will be added, this could include dialogue from NPCs, information from the signs on the floor that will tell you how to play the game or potential boss dialogue if I have time to implement it. The large gold boarder will mean the user can see clearly where to read and will attract their eyes the moment it pops up so they will put their full attention on it

**Checkpoint image**




Example background image (Not what my game will look like)

Rest option for character to regain health and flasks

Option to upgrade character with flames

Checkpoint pop-up box

There will also be a large checkpoint title at the top above the rest option to indicate what UI part they are looking at.

**Upgrade Image**




Health gets increased by + 10

Stamina gets increased by + 10

Attack damage gets increased by + 10

There will also be a large checkpoint title at the top above the rest option to indicate what UI part they are looking at.

### Input Validation

I have considered where input validation would be required through my game, however there is no need for any input validation as I do not need the user to input anything into the game other than simply pressing the keys to make the character move, but even then there is only one possible way they can get it to work therefore it is not necessary; this goes for most games on the market and it may only arise for certain things such as entering your characters name for a scoreboard. My game does not require anything to be physically inputted in terms of typing (correct data types or character lengths).

### Justification for approaches

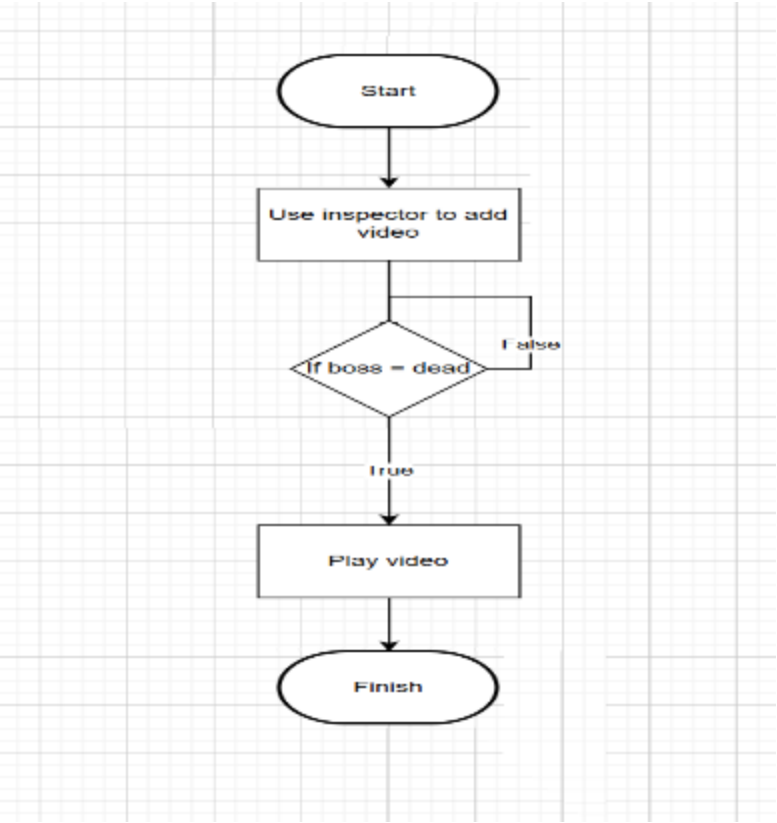
All the methods of identifying the components of my game and what they will look like will help me a lot to understand what needs to go where and how I can build off of the components chosen. This allows me to avoid making errors and saving great amounts of time fixing problems or having difficulties knowing what to do and how to do it.

## Algorithms

### Flowcharts for the broken-down problems

Using flowcharts will help me build my game in a modular and systematic way so I can solve each problem as it arises without getting creation points in the game mixed up.

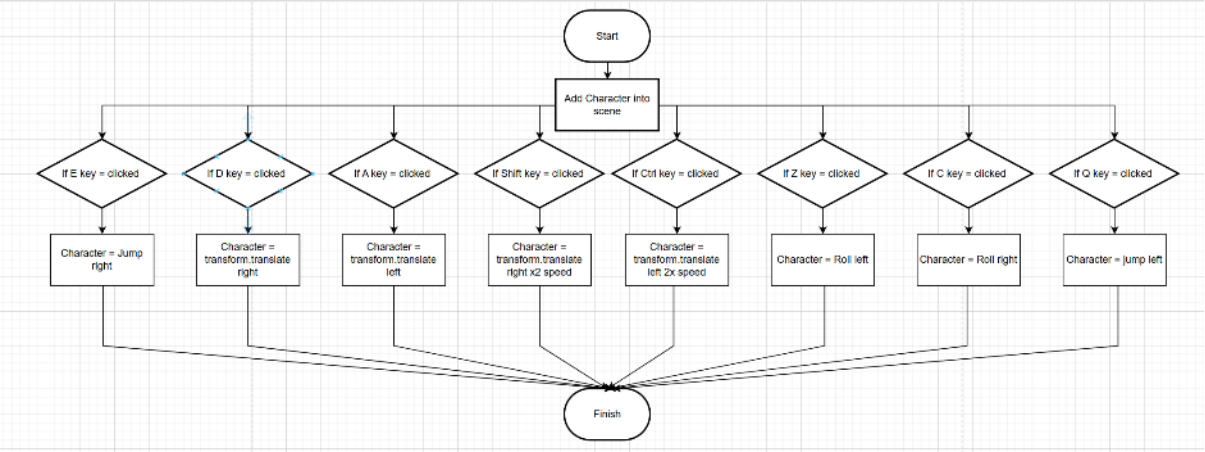
Cutscenes




This will be simple and easy to do with only a single if statement needed to work out if the boss is dead or not as the cutscenes will play after the boss is killed.

Character

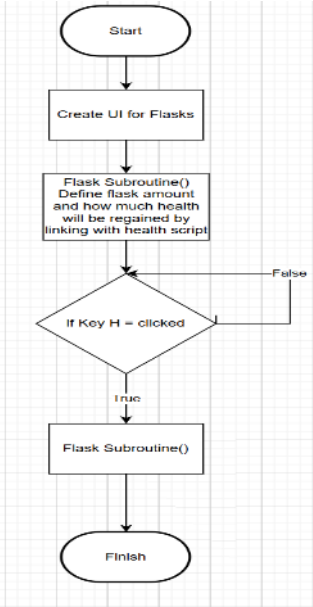
**Movement**




This will take time but hopefully will not be too difficult, the rolling and jumping will have to depend on more physics side as the character will have to roll with their momentum

and jump will have to angle the character upward to match the dark souls like game as the jump is not straight upward but more forward and slightly up.

**Flask**



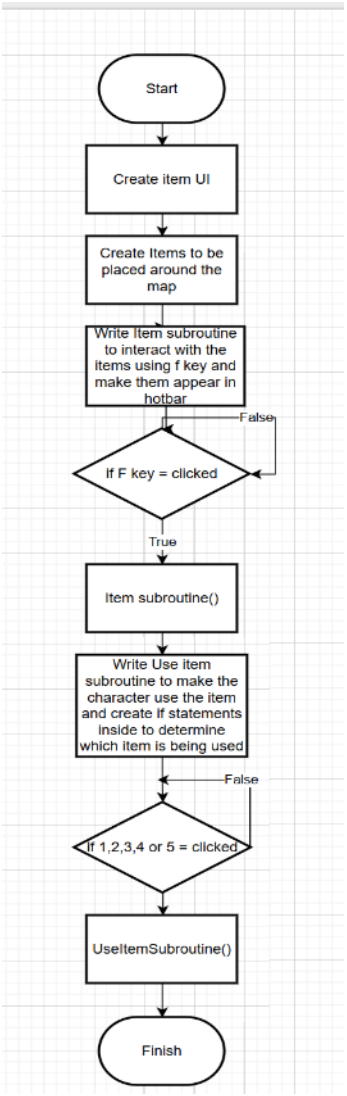

This shouldn't take too long or be difficult and will allow the character to regain health however this can only be done after the health script is created to allow for me to link it with the character and for it to make sense in my development path and success criteria.

**Attacks**




This will allow the character to damage enemies, I forgot to mention about declaring variables for the damage of the light and heavy attacks, this will have to come in after I have created the enemies and their health for the attacks to actually do anything.

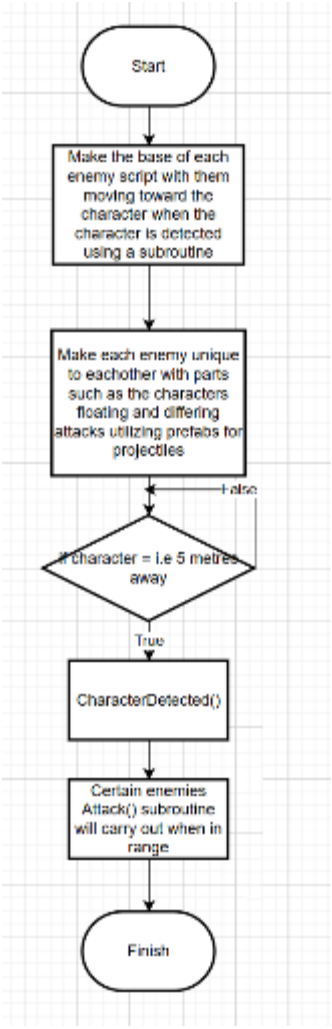
## Items




This will be a relatively long script and will take a large amount of time but shouldn't be extremely difficult to do, it will allow the player to do many things to help themselves throughout the game, increasing the strategy and playability.

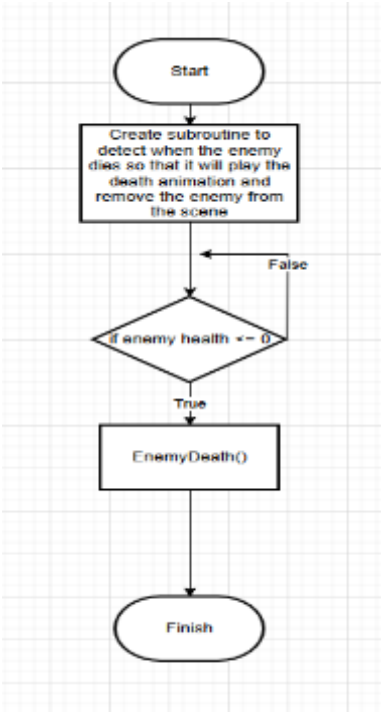
Enemies and Bosses

**Movement/ Abilities**



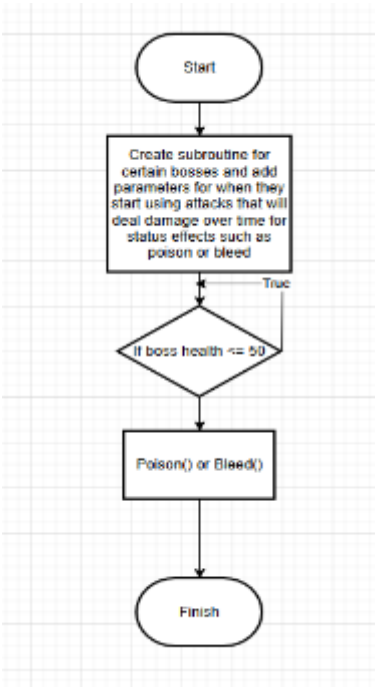

This will take a very long time but will make my game much more difficult and strategic meaning the enemies will all be different so the player will have to adapt to the changes of each enemy, each different enemy will be located in different areas of the game, increasing in difficulty as the game continues.

**Death Animation**




This will be short and easy piece of code to write and will help make the game more immersive, I will have to create the death animation in the unity inspector and then apply it using the animator built into unity.

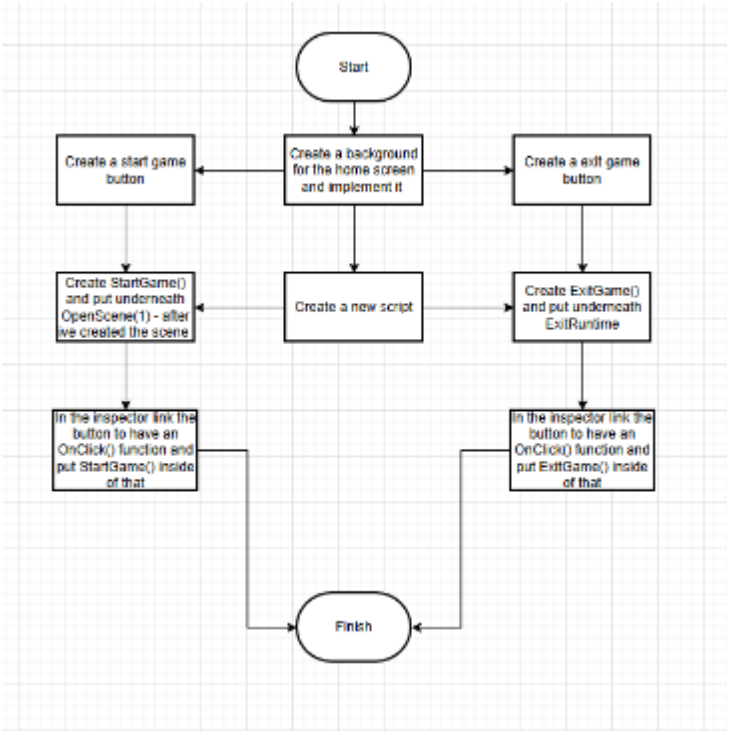
**Extras – If I have the time**




This will make the game more increasingly difficult and make the bosses feel like they are genuinely a challenge, I may add effects to the character to let the player know that they have a status effect active on them.

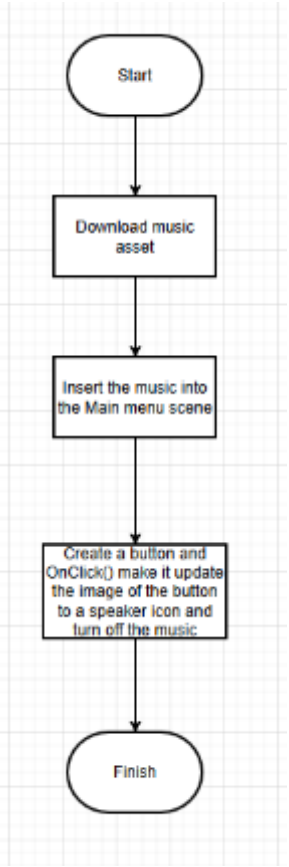
Main Menu

**Start and exit game buttons**




This will give the user UI to navigate and create an introduction to the game. This will not take long at all.

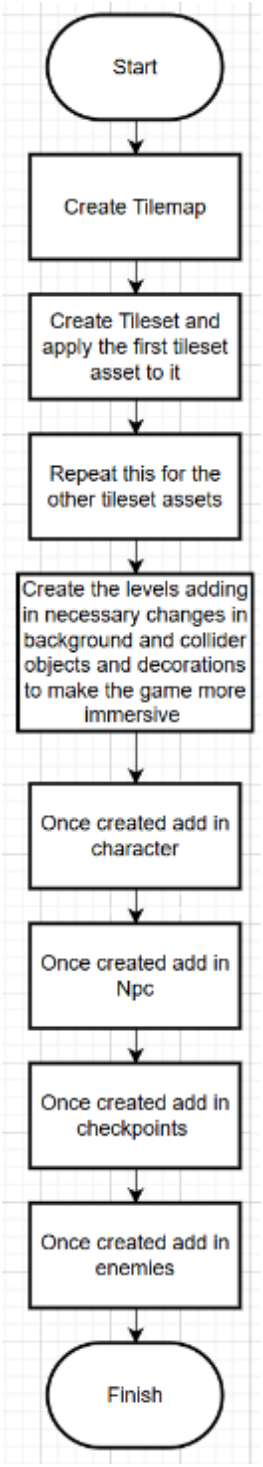
**Music Button**




This will allow the user to turn the music on and off making my game more interactive and give the user more to do.

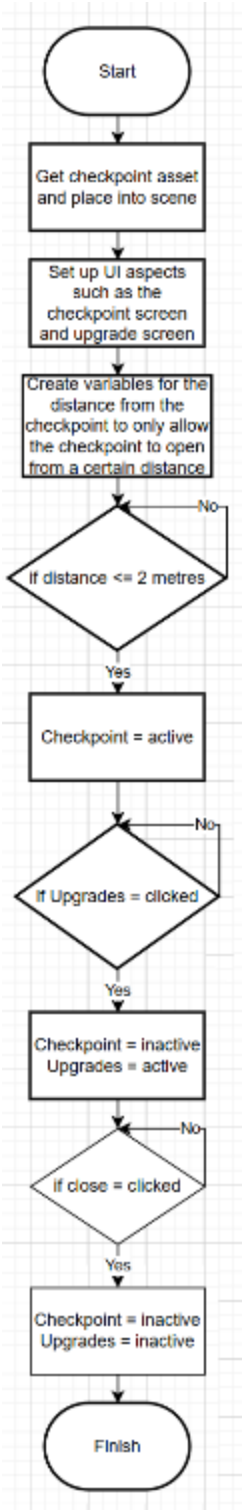
Level Design

**Creation of levels**



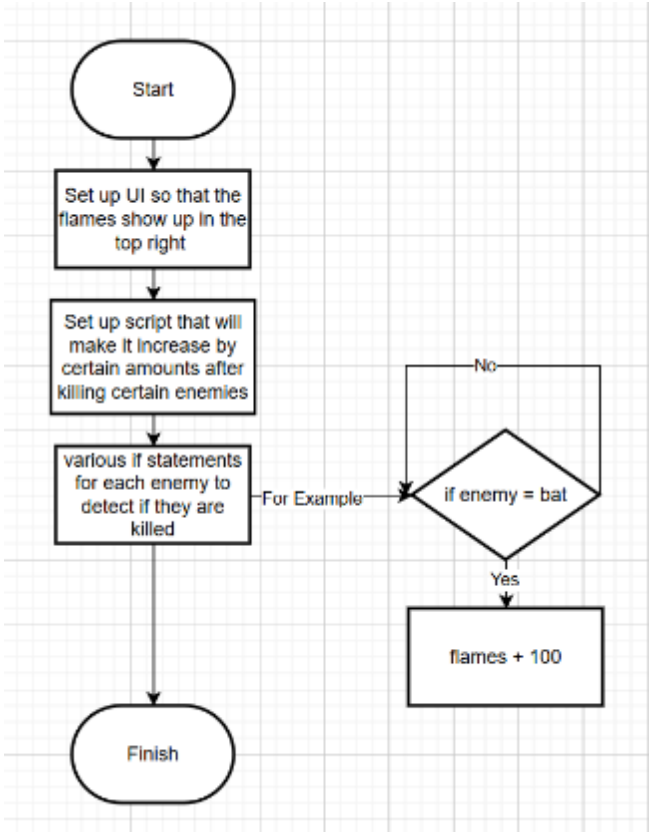

This potentially could take a long time as it will be long to create all the tilemaps and tilesets and format them in order for it to work with my level. It will make my game a lot more interesting and rewarding to play as it makes the player feel like they are genuinely progressing through the game.

Checkpoints




This will potentially take quite a long time but should not be incredibly difficult, the only way I can see it becoming difficult is the NPC interaction will use the same logic so could potentially interfere with each other so I will plan a workaround for that. This will give the user a more strategic gameplan and allow the character to upgrade health, stamina and attack damage. Once I have created the health, stamina and attack I will be able to add these into the checkpoint script so it can take effect.

Flames




This will not take long however setting the UI to update may deem difficult, I also will have to update the UI so that the flames will increase in cost for each one after each upgrade. This will make the game even more strategic and difficult for decision making.

I am able to follow these algorithms and turn them into code that I need to complete my project, this allows me to easily identify the next problem to be coded without having to worry that I am missing anything out of the final product.

# Data Structures

## Class diagrams

Bullet points = GameObjects defined from **Unity** and variables needed for script

Numbers = subroutines needed for the gameobjects and variables

**Main Menu**

- MainMenuCanvas
    1. Start()
    2. Update()
    3. StartGame()
    4. ExitGame()

**Character Movement**

- Animator
- SpriteRenderer
- moveSpeed
- jumpforce
- jumpCooldown
- rollSpeed
- rollCooldown
    1. Start()
    2. Update() – Everything should be able to be done using if statements inside this.

**Side Scrolling Camera**

- Character
- cameraOffset
- followSpeed
    1. Start()
    2. LateUpdate()

**Character Interactions**

- Npc
- Checkpoints
- checkpointScreen
- upgradeScreen
- interactionDistance
- displayDuration
- letterDelay
- isInteracting
    1. Start()
    2. Update()
    3. HandleInteractions() – only needed when checkpoints are added
    4. InteractWithNPC()
    5. DisplayMessages()
    6. DisplayLettersOneAfterAnother()
    7. CheckpointScreen()
    8. CheckpointExit()
    9. UpgradesScreen()
    10. UpgradeExit()

**Heart Health**

- maxHearts
- currentHearts
- heartPrefab
- heartContainer
- List (heartObjects)
    1. Start()
    2. Update()
    3. CreateHearts()
    4. TakeDamage()
    5. UpdateHearts()
    6. Heal()
    7. AddHeart()

**Stamina System**

- maxStamina
- currentStamina
- regenerate
- Slider (for stamina bar)
    1. Start()
    2. Update()
    3. RegenerateStamina()
    4. UseStamina()
    5. DrainStamina()
    6. UpdateStaminaBar()
    7. IncreaseMaxStamina()

**Items**

- ItemBar
- Item array
- StaminaUsed
- UseCooldown
    1. Start()
    2. Update()
    3. UseItem()
    4. PickUpItem()
    5. UpdateItemBar()

## Justification of the class diagrams

**Main Menu**

MainMenuCanvas – Needed to reference the main menu screen and access parts inside of the object

Start() – needed for parts that are going to be activated immediately i.e sound

Update() – needed for parts that are going to be activated later on

StartGame() – needed for when the game is going to be started

ExitGame() – needed for when the game is going to be exited

**Character**

Animator- needed to reference the animator so I can make the character perform animations

SpriteRenderer – needed for rendering and accessing parts of the character's sprite

moveSpeed – Sets the speed the character will move at

jumpForce – Sets the force at which the character will be propelled upwards

jumpCooldown – Sets the time between when the character will be allowed to jump again

rollSpeed – Sets the speed at which the character moves forward when they roll

rollCooldown – Sets the time between when the character will be allowed to roll again

Start() – for items being set at the start of runtime

Update() – for items being set after runtime has started i.e everything to do with the characters movement as it will not be necessary to have separate subroutines as I can simply repeat if statements

**Side Scrolling Camera**

Character – Needs to be referenced so the camera will follow the character

CameraOffset – Sets how far the character will move from the center of the camera before it will start following

followSpeed – Sets how fast the camera will catch up to the character as they move

Start() – needed for items being set at the start of runtime i.e cameraoffset

LateUpdate() – needed for item being set after runtime has started, delayed so the camera feels more fluid

**Character Interactions**

Npc – For when the npc gameobject needs to be referenced

Checkpoint – for when the checkpoint gameobject needs to be referenced

checkpointScreen – for when the checkpoint canvas needs to be referenced

upgradeScreen – for when the upgrade canvas needs to be referenced

interactionDistance – Sets how far away the character has to be from the npc of checkpoints to access them

DisplayDuration – How long text from the npc will stay up for

letterDelay – adds to the game like effect of the letters appearing one by one

Start() – needed for items being set at the start of runtime i.e checkpoint canvas being set to false

Update() – needed for items being set after runtime has started

HandleInteractions() – needed to determine if they are accessing checkpoint or npc

InteractWithNPC() – for the interaction with the npc/ what the npc will do when interacted with

Displaymessages() – for the messages appearing from the npc

DisplayLettersOneAfterAnother() – for the letters in the messages appearing one by one, will be referenced in the displaymessages()

Checkpointscreen() – for when the user interacts with the checkpoint

CheckpointExit()- For the user to exit the checkpoint screen and back to the game

UpgradeScreen() – for when the user opens the upgrade screen through the checkpoint screen

UpgradeExit() – for the user exiting the upgrade screen and back to the checkpoint screen

**Heart Health**

maxHearts – number of hearts that you can upgrade to

currentHearts – number of hearts the user is currently on

HeartPrefab – the heart object as a prefab so can be added in and removed easily during runtime

HeartContainer – where the hearts will be located on the screen and where they will be added in so hearts are not added in random locations

List HeartObjects -the list of heart prefabs added in, updating constantly

Start() – for items being set at the start of runtime i.e the starting amount of  hearts

Update() – for items being set after runtime has started

CreateHearts() – used to add a heart in on the screen

TakeDamage() – removes certain amount of hearts based at 1, but can be multiplied depending on the enemy i.e a boss will remove 2 hearts

UpdateHearts() – running number of hearts currently on the screen

Heal() – replenishes hearts

AddHearts() – adds a heart onto the max health aslong as it is under 7

**Stamina Bar**

maxStamina – sets the maximum amount of stamina the character has

currentStamina – the amount of stamina the character is currently on

regenRate – the rate stamina is replenished

staminaBar – reference the stamina bar game object

Start() – for items being set at the start of runtime i.e maxStamina being the current stamina

Update() – for items being set after runtime has started

RegenerateStamina() – for stamina being regenerated over time if it below the max stamina

UseStamina() – for large chunks of stamina being removed at a time on big movements i.e jumping

DrainStamina() – for small amounts of stamina being removed over time i.e for running

UpdateStaminaBar() – Shows the current value of the stamina on the stamina bar

IncreaseMaxStamina() – for upgrading the maximum amount of stamina the character can have

**Items**

itemBar – what the items will be shown/ visualized on – the game object

Item array – All the items available to be picked up as prefabs

StaminaUsed – Certain items will use stamina when activated

UseCooldown – All items will have a cooldown after they are used

Start() – For items being set at the start of runtime i.e an empty stamina bar

Update() – For items being set after the start of runtime

UseItem() – For when the player wants to use an item in the item bar which will be located using numbers on keyboard

PickUpItem() – For when the player wants to pick up an item

UpdateItemBar() – Refreshes the item bar after certain things happen i.e using an item or picking up an item

### Formatting

All script files will be in .cs (C# Source file)

All images will be in .jpeg or .png depending on where the image comes from. **Unity** is able to use both these formats

All tile sets will be in .png which then can be sliced and created inside of **Unity**

## Testing

### Testing Table for Success Criteria

Identifying milestones and tests that will need to be performed.

| No. | Milestone | Inputs | Expected outcome |
|---|---|---|---|
| 1. | Navigational home screen | Click start game<br>Click exit game<br>Click sound setting | Be able navigate the home screen with start game making a loading screen appear for a set amount of time and then game starts, exit game exiting the game and the sound setting turns sound on/off. |
| 2. | Levels | Multiple levels are present | There are many levels each with varying looks and should have |

| | | | |
|---|---|---|---|
| | | | cave – like backgrounds |
| 3. | Character movement | Q,W,E,A,S,D,Z,C, Shift and Control | All keys do their own respective functions creating a fully working character |
| 4. | Interactions | Press F on NPC or checkpoints and the NPC/ checkpoints are present on the level | The NPC will talk to you telling your fate, the checkpoints will show a navigational checkpoint screen and upgrade screen |
| 5. | Enemies and Bosses | The character goes near the enemy, enemies are located around the levels | The enemy will attack you, bosses will get a health bar at the bottom of the screen when in proximity, they will take damage if user attacks |
| 6. | Working Health and Stamina bars | Upgraded by clicking upgrade health/ stamina | Health and Stamina bars are present on the screen, when upgraded the amount of health and stamina is increased and can be visually seen by lengthening the bars |
| 7. | Extras – Not necessary but good additions to the game | Various inputs that can be seen after the health and stamina on success criteria such as H being for flasks | Working item bar, Flames decrease after spending on upgrades and increase after killing enemies, flasks are used to heal the character and show in the bottom left of the screen, character respawns at the last accessed checkpoint. |

## During/ Post development tests

These are the tests that I will carry out to validate/ invalidate all possible types of data inputs to ensure that the game works as intended with no bugs based off of the section above (Testing table for success criteria). These tests will take place during and post development to ensure the milestones completed after do not alter the previous ones. This could arise from certain gameobjects being changed or new code being added that may interfere with older code.

| Milestone | How will the milestone be tested? |
|---|---|
| Navigational home screen | Click each button and back button to ensure they work as expected and does not take the player to the wrong screens. Enter various keys on the keyboard even though the home screen is navigated with mouse. Click at various points on the screen to see if it opens anything where no buttons are. Click multiple buttons in quick succession to see if it will break. |
| Levels | Ensure that the levels are present on screen and that the character can navigate the obstacles easily without making the character stop. Attempt to find any possible ways to exit the level – i.e. holes in the floor/ ceiling – ways to glitch through walls. |
| Character movement | Ensure that each button on the keyboard works and does its expected function. Mash the keyboard with many inputs simultaneously to see if the character will break or not carry out animations/ behave in a not intended way. Press incorrect buttons to see how the game will handle them. |
| Interactions | Ensure the NPC will react in the intended way when you get near them and acts as intended when interacted with. Repeatedly press the interaction button |

| | |
|---|---|
| | to see if the NPC will not function correctly. Ensure the checkpoints will react as intended when you get near it and acts as intended when it is interacted with. In the checkpoints click around as various points to see if buttons interact even when not clicked. Click multiple buttons in quick succession to see if it will break. Enter keys to see if it will change anything. Make sure each button and back button works as expected. |
| Enemies and bosses | Ensure that the enemies/ bosses will work as intended, they will be alerted and attack the character when within a certain distance away from the character. I can use debug logs to identify how much health the enemies have taken to ensure the damage is being taken correctly. |
| Health and Stamina bars | Ensure that all the stamina decreases after movement and can be upgraded in the checkpoints. Ensure health will decrease correctly in proportion with the certain enemy and can be upgraded in the checkpoints. Spamming movement to see if the stamina will break. See if the game will end when health gets to 0. |
| Extras | Carrying out functionality, ad hoc, scripted, performance, regression and destructive testing for each extra addition to the game. |

**Testing Definitions:**

**Functionality** - Verifies that all game features work as intended.

**Ad Hoc** - Informal, random testing to find hidden bugs that might be missed by structured tests.

**Scripted** - Following a detailed test plan to systematically check specific parts of the game.

**Performance** - Focuses on how well the game runs by checking frame rates, loading times, and resource usage.

**Regression** - Used to ensure that a fix for one bug hasn't introduced new bugs elsewhere in the game. (Post Development)

**Destructive** - An effort to intentionally break the game to find its breaking points and vulnerabilities.

# Development

Throughout the development of my programs, I used **AI: ChatGPT and Copilot**. I used it as an aiding tool and every time I used it I made sure to tailor the code to how I needed it. Generally, the AI was always wrong with the code it provided for me and always needed extra alterations to it in order for the code to work.

## Assets

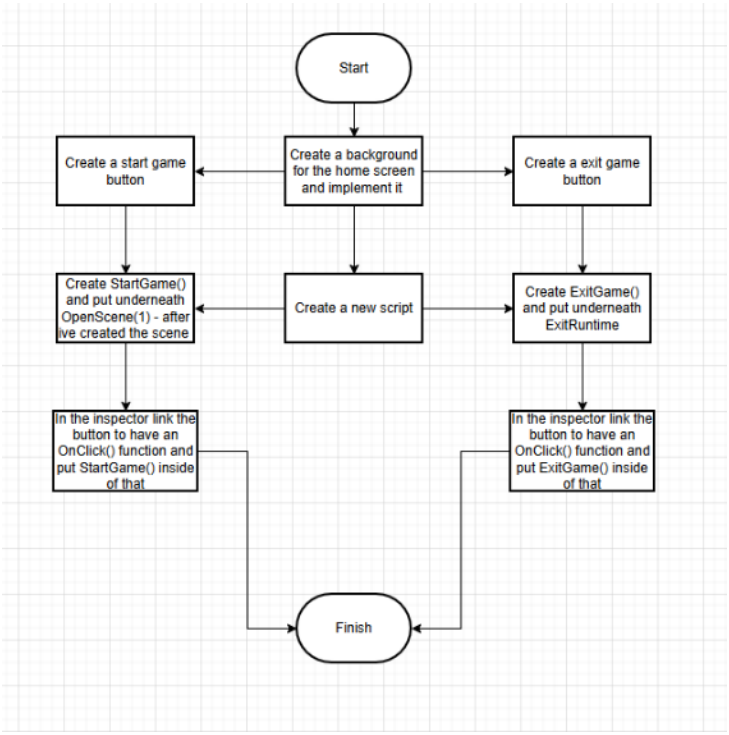
This assets table shows the assets that will be used, and what they will be used for. I will add to this asset table once I get round to doing the milestones that require more assets than I have shown in this table, For Example souls. Everything else in the game such as stamina bars I can create through paint or using the built in features of **Unity.** Certain images that I need can be created through **AI: ChatGPT** - **ChatGPT** gives the rights of the image created in the app to the user, allowing for them to do what they want with the image, including using it commercially. Using **ChatGPT** means I can get more specific images for what I need. Any images and assets I obtain will be through creative commons, being paid for or being distributed for free.

| Type | Asset Link |
|---|---|
| Character | https://free-game-assets.itch.io/free-fantasy-knight |
| Enemies | https://monopixelart.itch.io/dark-fantasy-enemies-asset-pack |
| Start Area | Oak Woods — Environment Asset by brullov |
| Scene One | https://maaot.itch.io/mossy-cavern |
| Scene Two | https://maaot.itch.io/fungal-grotto |
| Scene Three | https://maaot.itch.io/2d-browncave-assets |
| Music (Boss) | https://alkakrab.itch.io/6-dark-fantasy-boss-battle-tracks |

| | |
|---|---|
| Music (General) | https://alkakrab.itch.io/free-12-tracks-pixel-rpg-game-music-pack |
| Boss One | Necromancer (Free) by CreativeKind |
| Boss Two | https://creativekind.itch.io/nightborne-warrior |
| Boss Three | https://creativekind.itch.io/dark-soul-character |
| Checkpoint | Flying Obelisk by CreativeKind |
| Final Bonfire | RPG - SOULS LIKE BONFIRE by BlueBlade |

## Navigational Home Screen

I will be following this flowchart through and making any necessary improvements to my code during the programming.



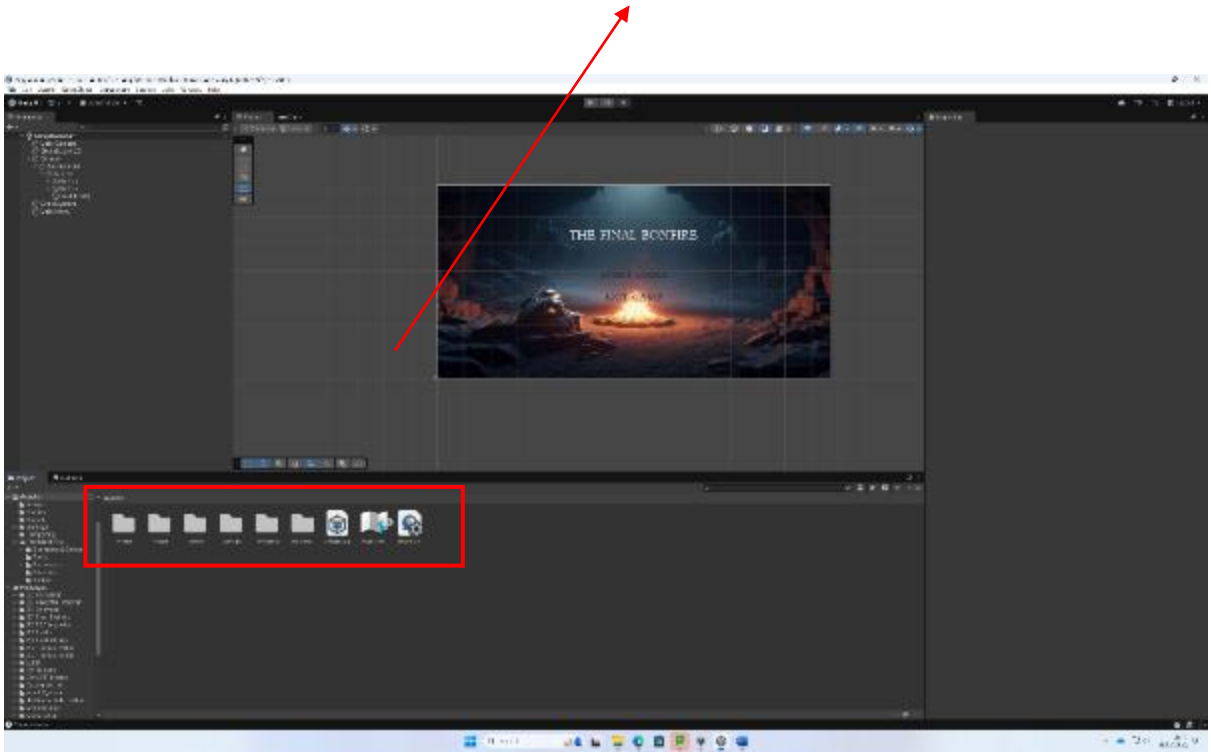
## Start and Exit game

Success criteria 1 – Start and Exit the game

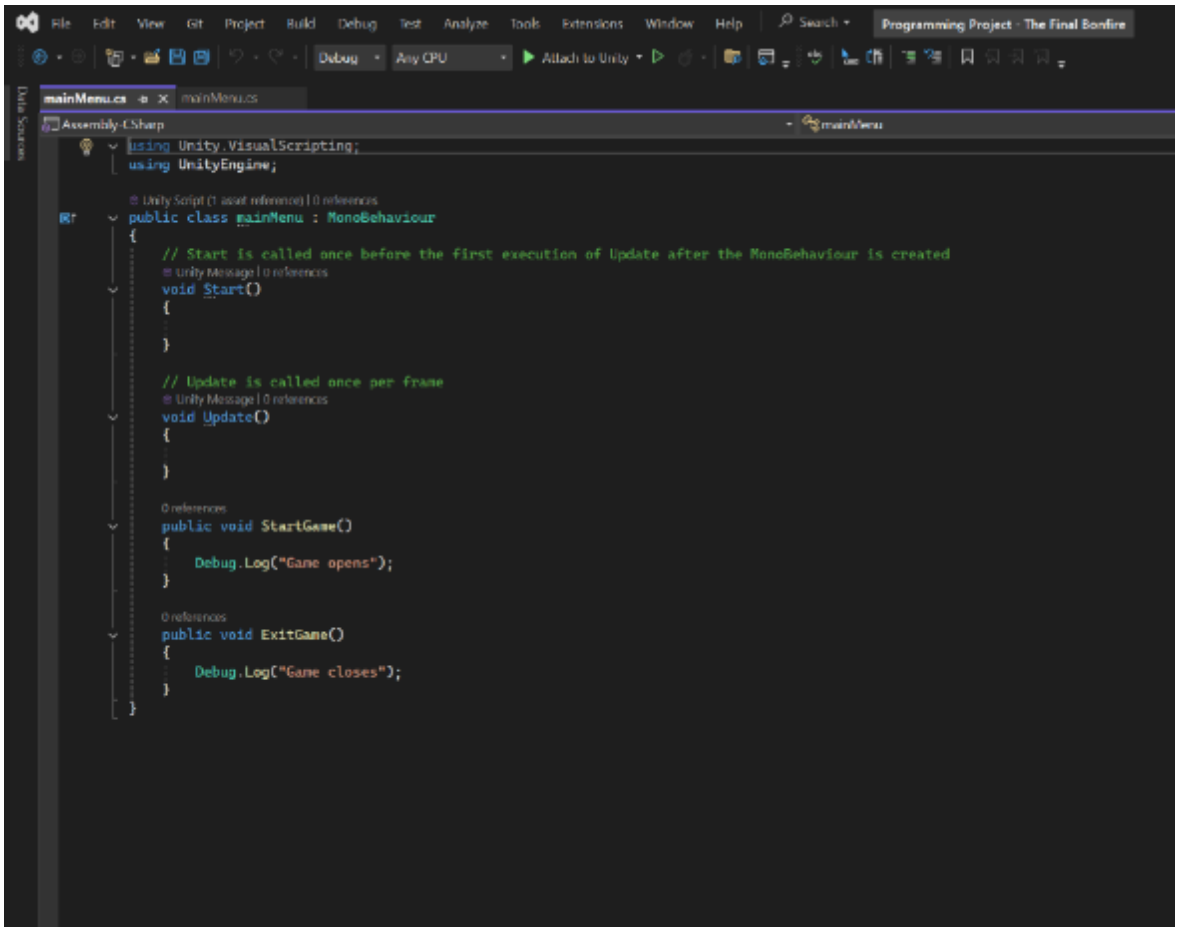
The main menu allows the user to start and close the game which is needed in order for the game to be run.

04/02/25

Added some folders for better file management




I started by adding the image which I obtained using **Chat GPT AI**, then added in text and two buttons for start and end game. After I added a different font file by changing the file format to textmeshpro within **Unity**.




Created a new script and added the two debug comments and applied the to the two buttons to show that the button does work how it is supposed to. This will later change to the “start game” sending you to the first scene and “exit game” closing the game.

## Sound

Success criteria 2 – Sound Setting

I quickly added in the sound by just adding in a audio source gameobject in the **Unity** inspector and applying my music, now every time I open the game it will start playing the music.

I added a sound setting on the menu by simply implementing another button which I included a sound icon for when it is enabled, in the inspector I set onclick to turn off the music.

**Testing table**

| Item | Expecting to see | Does it work? |
|---|---|---|
| Start game button | “Game starts” in debug log | Yes |

| | | |
|---|---|---|
| Exit game button | “Game closes” in debug log | Yes |

25/02/2025

After the first scene was created I implemented new code inside the start button so when it is pressed it will load the first scene.

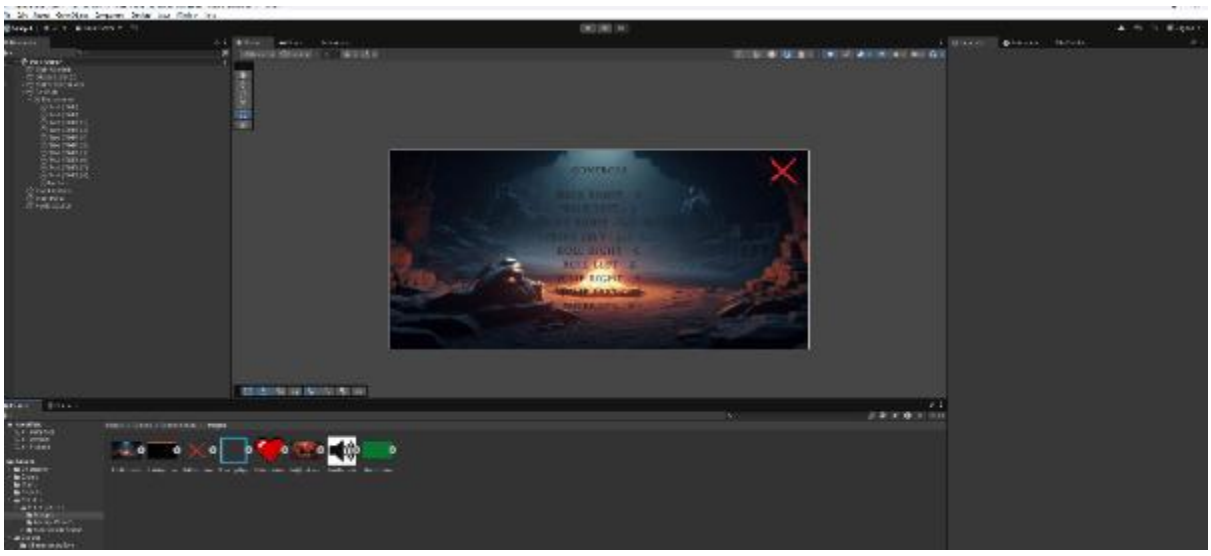
| | | |
|---|---|---|
| Start game button | Scene one opened | Yes |

## Controls Screen

(Not on success criteria)

19/05/2025

After review with the stakeholder I decided to add another screen into the main menu which would allow the player to view how to control the character, I labelled it controls and inside there would be text rows that would say i.e Walk left – A. This will allow the player to understand the game better and not be confused about how to control the character and also not miss anything out so all of the movements I provided and created would be used fully to their best ability. This was not in my success criteria but I felt it would be a useful addition and would not take long at all to do.



I started by making a Canvas (named Controls in the inspector) added the same image for the background and then simply entered multiple text objects and wrote out all the controls. After I quickly put in a new button, placed it in the top right, removed the text part and implemented the X image; I will later reference this in the code.

```
public class mainMenu : MonoBehaviour
{
    //reference gameobjects (Both Canvases)
    public GameObject MainMenuCanvas;
    public GameObject ControlsScreen;


    // Start is called once before the first execution of Update after the MonoBehaviour is created
    void Start()
    {
        MainMenuCanvas.SetActive(true);
        ControlsScreen.SetActive(false);
    }

    // Update is called once per frame
    void Update()
    {

    }

    public void StartGame()
    {
        SceneManager.LoadScene(1);
    }

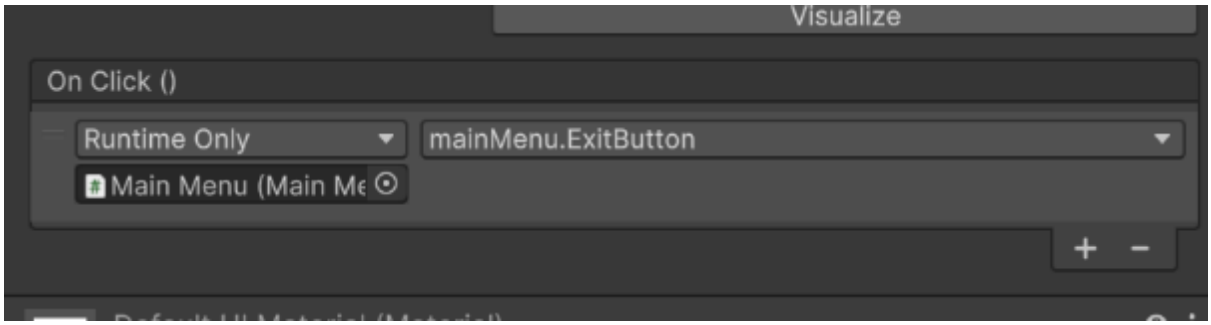
    public void ExitGame()
    {
        Debug.Log("Game closes");
    }

    public void Controls() // show the controls screen and turn off the main menu screen
    {
        ControlsScreen.SetActive(true);
        MainMenuCanvas.SetActive(false);
    }

    public void ExitButton() // show the main menu screen and turn off the control screen
    {
        ControlsScreen.SetActive(false);
        MainMenuCanvas.SetActive(true);
    }
}
```

I then implemented this code (StartGame() and ExitGame() stay the same) which I would be able to reference in the inspector to simply turn on and off either canvases when clicked.

For Example,




This is for when I click the exit button ( The red X ) it will play the exit button subroutine which will close the control screen and open the main menu screen

| Item | Expected to do | Does it work? |
|---|---|---|
| Control Screen button | Open the control screen | Yes |
| Exit Control Screen button | Close the control screen and open the main menu | Yes |

## Loading Screen

Success criteria 3

13/03/2025

I created a simple loading screen consisting of an image and a loading wheel, I speculated as to how this would actually work as unity doesn't exactly incorporate loading as all scenes are loaded at the same time and can be opened very quickly so there technically isn't any need for a loading screen, but to make the game seem more realistic I decided to add in a timer so that it will wait 5 seconds in the loading screen before opening the first scene. I ran into various problems to do with the loading wheel not being in layer with the image and going behind it so the wheel was not visible, I overcame this using another UI image element as I figured out that UI elements show themselves over an other object inside of the game. To tackle the timer problem I used a simple line of code to wait 5 seconds before opening the next scene as seen in the screenshot below.

```
IEnumerator TransitionToNextScene()
{
    yield return new WaitForSeconds(5f);
    SceneManager.LoadScene(2);
}
```

| Item | Expected to do | Does it work? |
|---|---|---|
| Loading screen | Wait for 5 secs then new scene opens | Yes |

## Review of the home screen

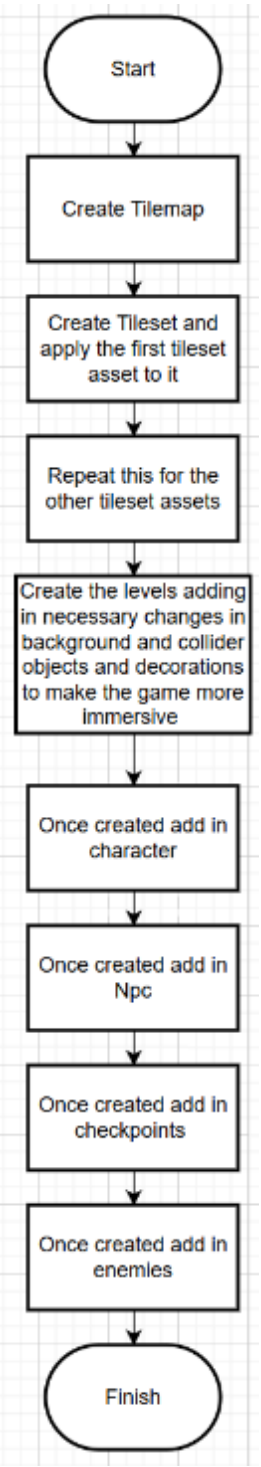
Black = Question  Red = Answer

- What do you enjoy about the home screen?
  I like the layout of the home screen, its simplistic and shows only what you really need to be on the screen. The music button is good as it gives me the freedom to decide if I want the music on or off. The loading screen is good, and the fact there is a loading wheel to indicate the game is loading and not frozen is good.
- What can I improve about the home screen?

To improve the home screen, I think adding a sound effect when clicking each button would be good as it would indicate that the button has been clicked and could add a greater atmosphere to the game.

After this review I decided to take the stakeholders advice and add a sound effect to the buttons. To do this I simply added an extra addition to the OnClick() function that is built into unity and added a sound effect to it. I think this is a good addition to the game.

## Levels

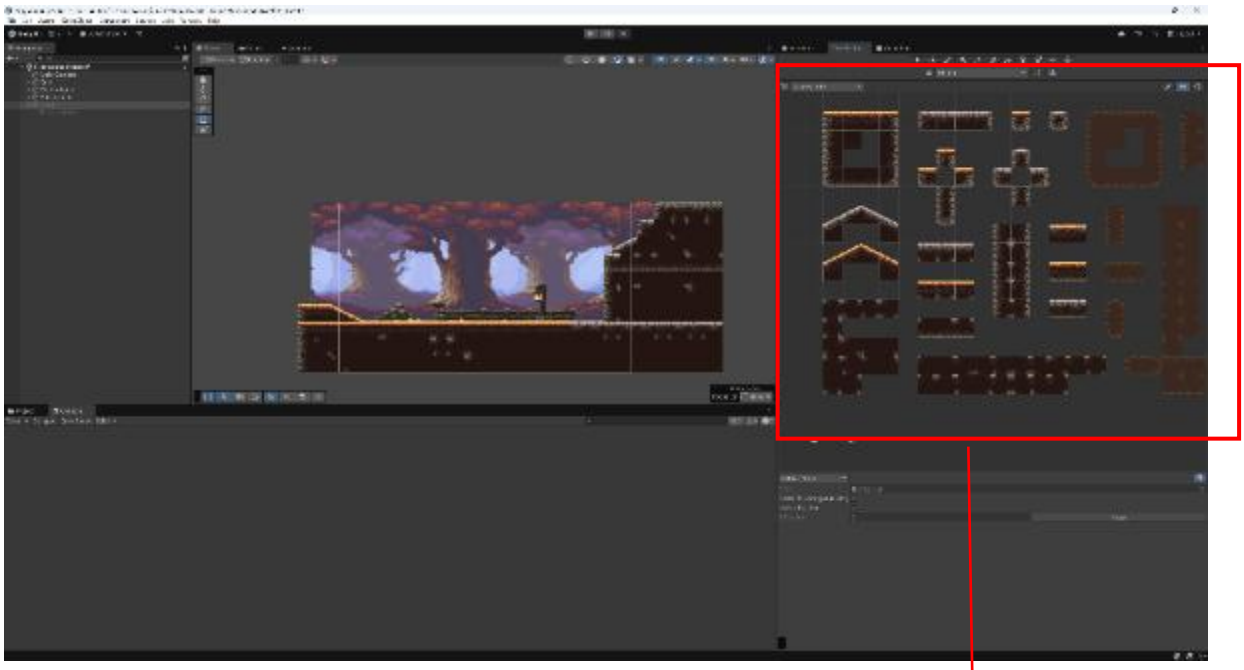



I will be following this flowchart and make any necessary changes as I create the levels.

Success criteria 4

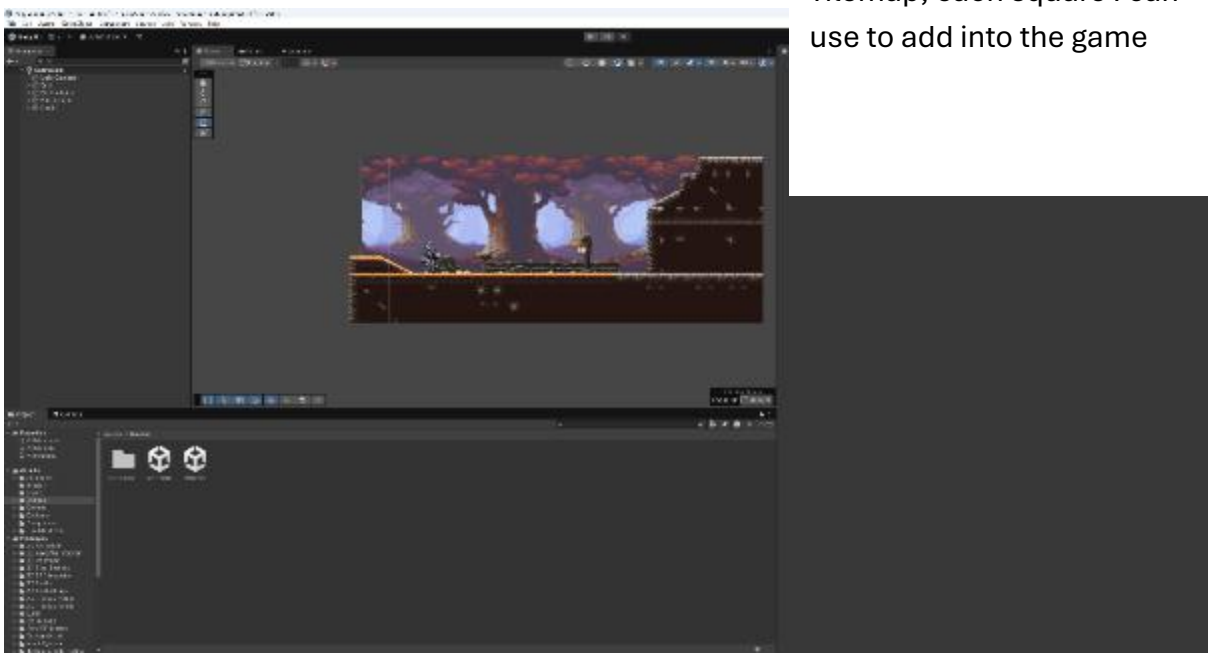
25/02/25

I created a basic first scene (seen below) so I could get started on my character, I did this by utilizing **Unity's** sprite editor and tilemap object which allowed me to split the given tilemap provided in the scene pack. I then added in a background by using multiple background object layers.



After I created the character I added him in.

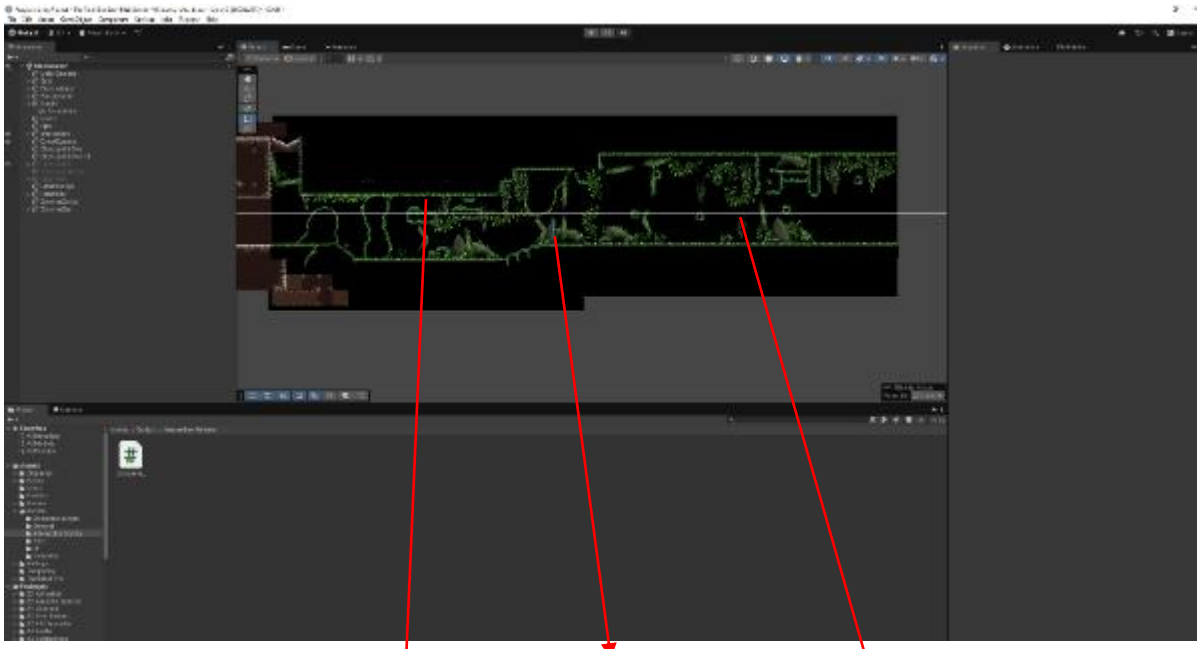
Tilemap, each square I can use to add into the game



02/05/2025

I added in the next part of the scene by extending the cave into a large mossy cavern cave using the mossy cavern tileset. I did the same process as before adding in a floor/

solid object tilemap and background tilemap, using various decorations and flooring I created a small level and boss room at the end. This will increase the amount the player has to play and make the gameplay more adaptive and immersive.




## Review of the levels

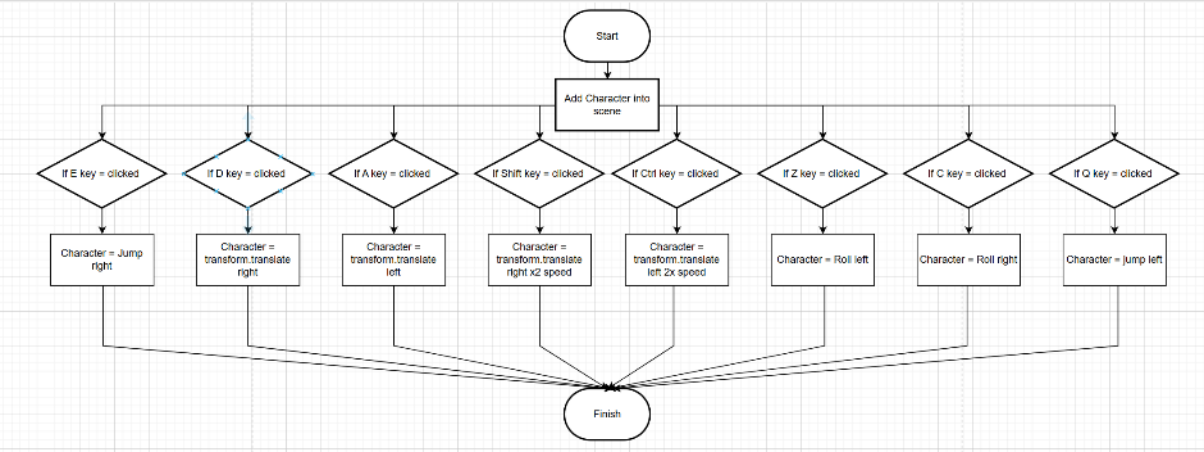
Black = Question  Red = Answer

- What do you enjoy about the level(s)?
  I think the level at it's current state is good, the design is good and simplistic, which is what you need for the first level in the game. The straightforward look of it allows me to focus on what will be the enemies, and the environment not being very dense but detailed means that I will be able to move through without major distraction from what I will be fighting.
- What could I improve about the level(s)?
  At some parts, I am able to see the grey areas of where black background ends, so fixing this would make the game more immersive.

After reviewing the stakeholder feedback, I agree with the comment on the grey areas of the background, therefore I simply just added some more black background in order to make sure the user couldn't see the grey areas.

## Character Movement

I will be following using this flow chart from the design and changing my pseudocode into actual code.




25/02/25

### Idle, Walking, Running

Success criteria 5

At the start of creating the character, I added in the sprite and using unity's animator tool I animated the idle, walk and run animations that were provided with the character pack. Then I added code to make the character walk/ move slowly when D is pressed and run/ move quickly when Left-Shift is pressed.

```
{

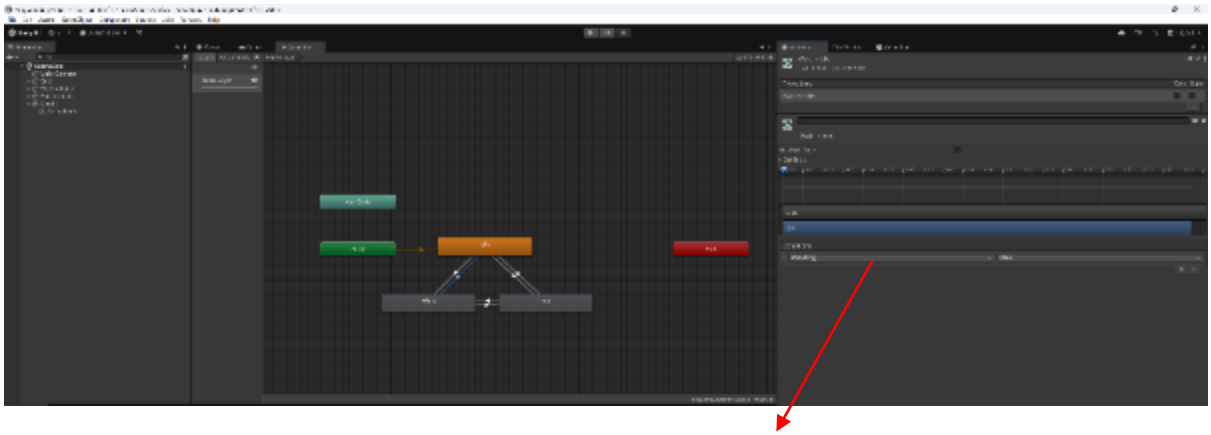
    if (Input.GetKey(KeyCode.D))
    {
        moveSpeed = 0.5f;
        transform.Translate(Vector2.right * moveSpeed * Time.deltaTime);
```

```
if (Input.GetKey(KeyCode.LeftShift))
{
    moveSpeed = 1f;
    transform.Translate(Vector2.right * moveSpeed * Time.deltaTime);
```

**Testing table**

| Item | What is expected? | Does it work? |
|---|---|---|
| Walk | Move slowly | Yes |
| Run | Move quickly | Yes |

After, I had to get the animations working. I created an animator chart using the animator tool in **Unity**, which allows the animations to shift based on parameters I declare.

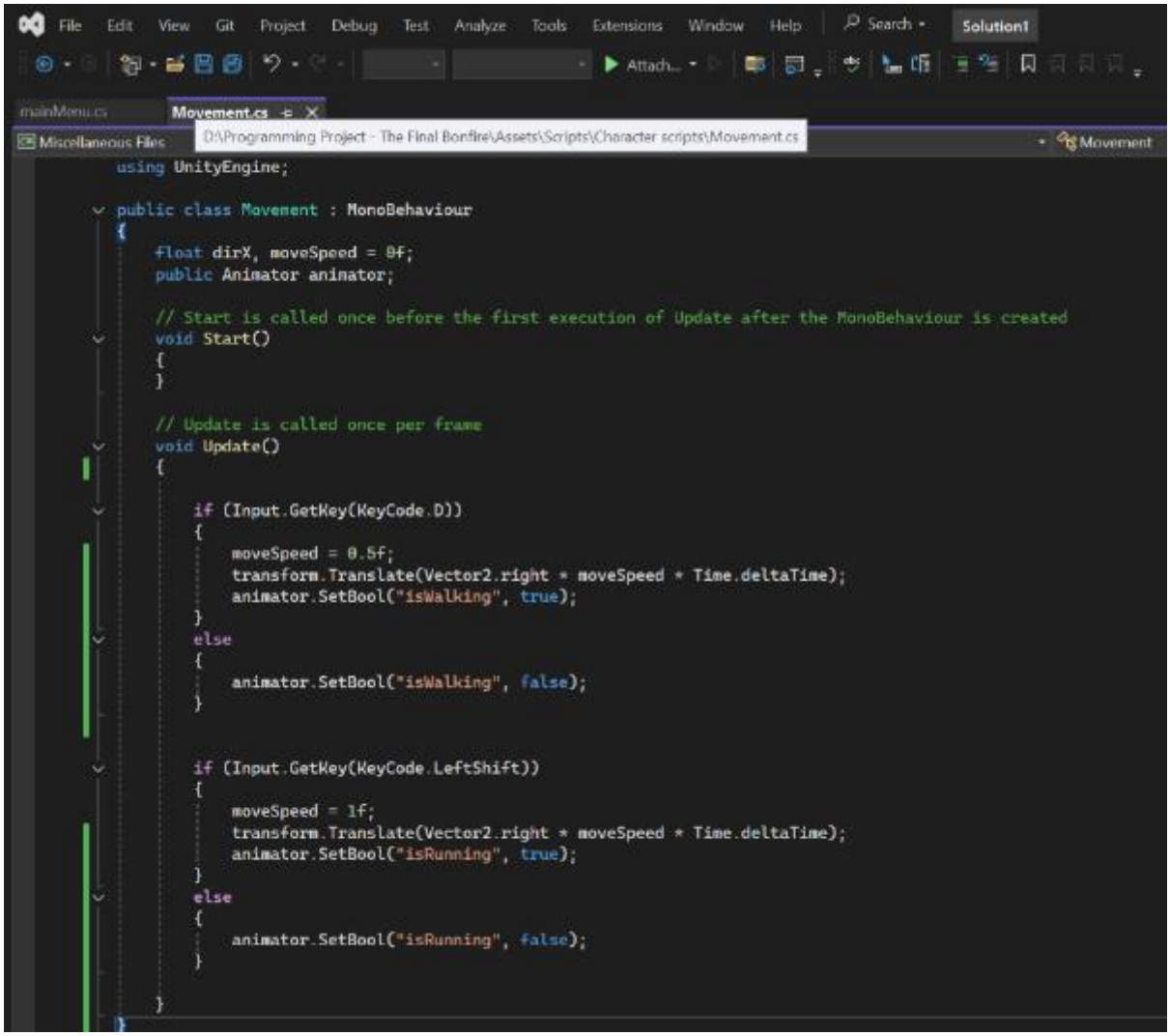


Parameter (isWalking)

After I attempted to implement this into the code, I had various problems and errors throughout this task.

| **Item** | **What is expected?** | **Does it work?** |
|---|---|---|
| Walk animation | When D is pressed the character walks and the animation works | Yes |
| Run animation | When Shift is pressed the character runs and the animation works | No |
| Idle animation | When there is no input idle animation works | No |

This took a lot longer than I had originally expected but in the end I worked out a simple few lines of code that allowed it to work.




In the code I have to allow the parameters to be set to true when the key I want is pressed, and after it is no longer being pressed the 'else' makes the animation be set to false so the character stops.

**Testing table**

| Item | What is expected? | Does it work? |
|---|---|---|
| Walk animation | When D is pressed the character walks and the animation works | Yes |
| Run animation | When Shift is pressed the character runs and the animation works | Yes |
| Idle animation | When there is no input idle animation works | Yes |

In order for the character to walk/ run backwards I just had to rewrite the code and change the 'Vector2.right' to be 'Vector2.left' to cause the character to move in the

opposite direction. Then to make the character sprite turn I added a transform.localScale command and changed the x direction to a negative. Below is the code.

```
transform.Translate(Vector2.left * moveSpeed * Time.deltaTime);
animator.SetBool("isWalking", true);
transform.localScale = new Vector2(-0.75f, 0.75f);
```

This is what the movement looks like currently.



## Jumping

**06/03/2025**

I started working on adding the jumping mechanic, there were few problems with the code but I ran into difficulties when attempting to make it so the character simply jumps forward at the press of the space key like in **Dark Souls**. Below is the code I came up with.

```
//Jump
if (Input.GetKeyDown(KeyCode.Space))
{
    animator.SetBool("isJumping", true);
    float angleInRadians = jumpAngle * Mathf.Deg2Rad;
    Vector2 jumpDirection = new Vector2(Mathf.Cos(angleInRadians), Mathf.Sin(angleInRadians));
    rb.linearVelocity = jumpDirection * jumpForce;
    transform.localScale = new Vector2(0.75f, 0.75f);
```

In order to do this, I had to create an angled jump which can be seen where I've utilised the angleInRadians , JumpDirection and jumpForce variables.

This all worked fine after slight tweaking of the jump force. To make the character jump in the opposite direction I simply set the 'Mathf.Deg2Rad' and 'jumpDirection' to be negative as well as changing the first 0.75f (x-axis) to negative.

However, the animation does not play when I press the space key.

| Item | What is expected | Does it work? |
|---|---|---|
| E key is pressed | Jump to the right | Yes |
| Q key is pressed | Jump to the left | Yes |
| E/ Q key is pressed | There is a jump animation | No |

14/03/25

After various attempts and changing aspects of the code and in unity's animator controller tool I managed to get it to work by using a trigger function instead of using Boolean parameters to start the animation. The code below is what I added for it to work.

```
animator.SetTrigger("Jump");
```

Then I changed the exit time of the animation so it would only run the animation once instead of repeating constantly.

| Item | What is expected | Does it work? |
|---|---|---|
| E key is pressed | Jump to the right with jump animation | Yes |
| Q key is pressed | Jump to the left with jump animation | Yes |

I added a timer cooldown as I ran into some problems to do with the character falling over onto their side which I think is due to the angle of the jump which is fine after one jump but another in quick succession causes it to fall over so the timer solves the problem by giving the character a chance to stand straight before another jump.

```
if(Time.time >= nextJumpTime)
{
    if (Input.GetKeyDown(KeyCode.E))
    {
        float angleInRadians = jumpAngle * Mathf.Deg2Rad;
        Vector2 jumpDirection = new Vector2(Mathf.Cos(angleInRadians), Mathf.Sin(angleInRadians));
        rb.linearVelocity = jumpDirection * jumpForce;
        animator.SetTrigger("Jump");
        transform.localScale = new Vector2(0.75f, 0.75f);
        nextJumpTime = Time.time + jumpCooldown;
    }
    else if (Input.GetKeyDown(KeyCode.Q))
    {
        float angleInRadians = jumpAngle * -Mathf.Deg2Rad;
        Vector2 jumpDirection = new Vector2(Mathf.Cos(angleInRadians), Mathf.Sin(angleInRadians));
        rb.linearVelocity = -jumpDirection * jumpForce;
        animator.SetTrigger("Jump");
        transform.localScale = new Vector2(-0.75f, 0.75f);
        nextJumpTime = Time.time + jumpCooldown;
    }
```

This checks if the time waited is equal to or longer than the time before the next jump is possible. After the jump has happened the jumpCooldown is added to the time so that the player will have to wait until the jump cooldown has been finished.

This is what the jump looks like now.

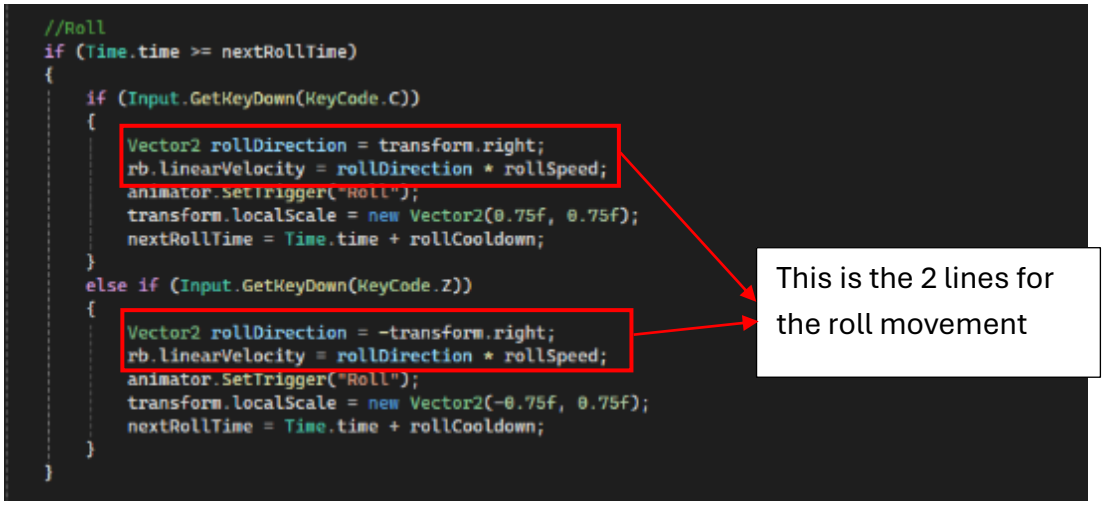


## Roll

**14/03/25**

Creating the roll was quite simplistic only using 2 lines of code for the actual movement, this is because I only needed it to move forward by a certain amount at the click of a button as in **Dark Souls** the roll mechanic is only one button and you just roll forward. There is a big mechanic in the game though which allows you to be invincible during the time of the roll, this is one of **Dark Souls** main signature mechanics which

many games have incorporated since, I will add this when it comes to it after I have created enemies so I can test it properly. Below is the code I've used.




The first line is to just dictate what direction the character will roll. The second line utilizes velocity which will be created from the direction the character rolls and multiplied by the Speed of the roll. The velocity aspect will cause the character to slow down after the button has been clicked and does not simply just carry on moving.

As you can see I have also added a timer cooldown for the roll as I was encountering the same problem with the flipping and falling over.

| Item | What is expected | Does it work? |
|---|---|---|
| C key is pressed | Character rolls right with correct animation | Yes |
| Z key is pressed | Character rolls left with correct animation | Yes |
| Timer | After each roll it waits 1.5 seconds | Yes |

This is what the roll looks like now.



### Review of the character movement

Black = Question  Red = Answer

- What do you enjoy about the movement?
  The movement feels smooth and fast which is what I like about 2D games, it also allows for faster paced gameplay making the game a lot more fun and exciting.
- What can I improve about the movement?
  The animations of the character could be more seamless as it makes the character look like it doing the wrong thing when the animations don't update in time.
- Is there anything I can change about other parts of the game to make the movement better?
  I had to work out all of the controls by myself upon entering the game so I think it will be beneficial to add a control screen in the main menu or having another way to inform the player of how to move the character and make sure they don't miss out on any features.

After this review I went back through the animations and made the transitions smoother by changing the exit time of them to 0 so that they blended better to different ones; as the exit time being longer caused the animation to carry on longer making the changes look weird. I updated the GIFs to show the new animations instead of the old ones. I also added in a control screen where I mentioned the review in the Home Screen section.

## Health and Stamina Bars

### Consultation about health script

Black = Question  Red = Answer

- What kind of health bar should I use?

I think using hearts will be the best fit for the game as I believe it makes it easier to understand how much health you have but also makes the game more intense and risky due to hearts creating a greater sense of urgency as you tend to not have many.

- How much health should the character have?
  I think 7 hearts is the best number of hearts/ hit points for this game as it will be harder and more strategic therefore causing the game to become more immersive and realistic.
- Should health replenish over time?
  I don't think health should replenish over time as it wont fit well with the heart system and will also end up making the game too easy.